

מכללת בסמ"ח

סמל מכללה: 71605 · המכון הממשלתי להכשרה טכנולוגית — מה"ט



מערכת לניהול סטודיו כושר ובריאות

פלטפורמת SaaS רב-דיירית (multi-tenant) לניהול סטודיואים — תלמידים, חוגים, מערכת שעות, נוכחות, עמלות ותשלומים מקוונים

טכנאים מוסמכים

מסלול הכשרה

הנדסת תוכנה

מגמת לימוד

Classly — מערכת לניהול סטודיו

נושא פרויקט הגמר

אופיר גואטה · איתי בר

שם המגשים

מר עילי גילת

שם המנחה האישי

יוני 2026

תאריך הגשת ספר הפרויקט

חתימת הגורם המקצועי

חתימת המנחה האישי

חתימת הסטודנט

תוכן עניינים

3	הצעת פרויקט	1
12	הצהרת הסטודנטים	2
13	הוקרות	3
14	תקציר	4
15	עקרונות בסיסיים בפרויקט	5
17	תרשים ERD	6
19	צד שרת	7
30	צד לקוח	8
42	תהליכים מרכזיים במערכת	9
47	סיכום ומסקנות	10
48	ביבליוגרפיה	11

1. הצעת פרויקט גמר

בהתאם להנחיות חוזר מנהל מה"ט 11-4-51, נספח מס' 1

תאריך: 07/12/2025

לידיעת: יחידת הפרויקטים — מה"ט.

1. פרטי הסטודנטים

שם מלא	תעודת זהות	כתובת מגורים	טלפון נייד	תאריך סיום לימודים משוער
אופיר גואטה	326091337	רזניק דויד 7, נתניה	058-6262800	2026
איתי בר	208731324	ביאליק 10, בית שמש	051-2087313	2026

2. פרטי המכללה

שם המכללה	בסמ"ח
סמל המכללה	71605
מסלול ההכשרה	טכנאים מוסמכים
מגמת לימוד	הנדסת תוכנה
מקום ביצוע הפרויקט	בסמ"ח

3. פרטי המנחה האישי

שם המנחה	עילי גילת
כתובת	האירוסים 67, קדימה צורן
טלפון נייד	052-6010916
תואר	מדעי המחשב, מכללת נתניה
מקום עבודה / תפקיד	צה"ל

* דיפלומת המנחה מצורפת בסוף המסמך.

4. שם הפרויקט

Classly — פלטפורמת SaaS רב-דיירית לניהול סטודיו כושר ובריאות.

5. רקע

עולם החוגים בישראל צומח ומתפתח משנה לשנה. יותר ויותר אנשים מחפשים מקום שבו יוכלו לשלב פעילות גופנית, אמנות ותנועה בחיי היומיום. עם זאת, מאחורי הקלעים מתנהל תהליך מורכב, ידני ומפוזר. הדבר יוצר קשיים הן מצד התלמידים, הנדרשים להתעסק ברישומים ותשלומים מסורבלים, והן מצד המורים והמנהלים, הנאלצים להשקיע זמן רב בארגון במקום להתמקד בהוראה ובפיתוח הקהילה.

תהליכים אלו גוזלים זמן יקר, פוגעים ביעילות ומקשים על כלל המשתתפים. מעבר לכך, פעילות ספורטיבית ואמנותית אינה רק עניין של בריאות פיזית — היא משפיעה ישירות על הנפש, יוצרת תחושת שייכות ותורמת לחוסן החברתי. לכן יצירת מערכת דיגיטלית מאוחדת שתפשט ותייעל את כל תהליכי הניהול בסטודיו היא לא רק פתרון טכנולוגי, אלא גם מענה חברתי וקהילתי חשוב.

6. מטרות המערכת

המערכת נועדה לספק פתרון דיגיטלי מלא לניהול חוגים וסטודיו — החל מהרשמות ותשלומים, דרך ניהול מערכות שעות ועד מעקב נוכחות. המטרה היא לאחד את כל התהליכים תחת פלטפורמה אחת, נוחה, יעילה ושקופה עבור מנהלים, מורים ותלמידים, תוך **בידוד נתונים מלא בין סטודיואים** (כל סטודיו הוא "דייר" מבודד שרואה רק את הנתונים שלו).

7. סקירת מצב קיים בשוק ובעיות קיימות

כיום, ניהול חוגים מתבצע לרוב בדרכים חלקיות ומסורבלות:

- רישום ותשלומים ידניים או בהודעות פרטיות.
- מערכות שעות שאינן מרוכזות וקלות לשינוי ללא תיאום.
- חוסר שקיפות בתשלומים, הן מול הסטודיו והן מול המורים.
- היעדר כלי דיגיטלי לניהול נוכחות.

התוצאה: עומס על המנהלים, חוסר סדר, טעויות אנוש ותחושת בלגן הפוגעת ביעילות הסטודיו ובחווית המשתמש.

8. מה הפרויקט אמור לחדש או לשפר

- **אוטומציה של תהליכים** — רישום ותשלומים מקוונים, וניהול מערכת שעות ונוכחות בצורה מרוכזת.
- **שיפור חוויית המשתמש** — ממשק אינטואיטיבי ופשוט לתלמידים, מורים ומנהלים, בעברית ובכיווניות RTL.
- **ייעול הניהול הפנימי** — פלטפורמה אחת לכלל הצרכים במקום שימוש בכלים מפוזרים.

- **תרומה קהילתית** — חיזוק תחום האמנות והספורט, עידוד פעילות גופנית ויצירת חיבור בין אנשים.

9. דרישות מערכת ופונקציונליות

9.1 דרישות מערכת (לא-פונקציונליות)

פירוט	דרישה
הפרדה מלאה בין נתוני סטודיואים לפי <code>studio_id</code> בכל שאילתה.	רב-דיירות ובידוד
תמיכה במשתמשים מרובים במקביל (עונות רישום וימי תשלום); שרת חסר-מצב הניתן להרצה במספר עותקים.	זמינות וביצועים
עדכון מהיר של מערכת השעות תוך שמירה על עקביות לכל המשתמשים; טרנזקציות בפעולות מרובות-טבלאות.	עקביות מידע
נגישות מכל מכשיר (מחשב, טאבלט, סמארטפון); ממשק עברי <code>RTL, mobile-first</code> .	נגישות ושפה
שמירת מידע רגיש (פרטי תלמידים ותשלום) באופן מאובטח — <code>JWT, bcrypt, HTTPS</code> .	אבטחה
הרצה מלאה תחת <code>Docker Compose</code> — בסיס נתונים, שרת, צד לקוח ולוגים.	תשתית אחידה

9.2 דרישות פונקציונליות לפי תפקיד

מסך התחברות — כולל הרשמה והזמנת משתמשים (מנהל / מורה / תלמיד / מנהל-על).

צד מנהל (ADMIN)

- ניהול סניפים, חדרים, קטגוריות וחוגים
- ניהול תלמידים והרשמות
- ניהול תשלומים (לסטודיו ולמורים)
- בנייה וניהול מערכת שעות מרכזית
- צפייה בדוחות ומדדים (נוכחות, כספים, הרשמות)

צד מורה (INSTRUCTOR)

- מערכת שעות אישית
- רשימת תלמידים לכל שיעור
- סימון נוכחות לכל מפגש

צד תלמיד (STUDENT)

- עיון בחוגים והרשמה מקוונת
- תשלום אונליין מאובטח
- צפייה בהרשמות ובלוח האישי

מנהל-על (SUPER_ADMIN)

- ניהול חוצה-סטודיואים והגדרות פלטפורמה
- מדדים ובקרה ברמת המערכת כולה

10. בעיות צפויות במהלך הפיתוח ופתרונות

10.1 בעיות צפויות

- עומסים על המערכת בתקופות רישום ותשלום.
- ניהול שינויים במערכת שעות העלול לבלבל תלמידים ומורים.
- שמירת מידע רגיש (פרטי תשלום ותלמידים).
- ניהול מספר סטטוסים (הרשמה/תשלום) במקביל ושמירת עקביות.
- בידוד מלא בין נתוני סטודיואים שונים.

10.2 פתרונות

- **שרת חסר-מצב (stateless)** הניתן להרצה בכמה עותקים מאחורי Nginx, וקונטיינרים עם `restart` policy תחת Docker — להפעלה מחדש אוטומטית בעת תקלה.
- **טרנזקציות** (`prisma.$transaction`) לשמירת עקביות בפעולות מרובות-טבלאות, ואידמפוטנטיות ב-**Webhook** למניעת עיבוד כפול של תשלום.
- **אבטחת מידע** — הצפנת סיסמאות (bcrypt), JWT בעוגיית HttpOnly, תקשורת HTTPS (דרך Cloudflare), ובידוד לפי `studio_id`.
- **ולידציה וניקוי קלט** (Zod + sanitize-html) ובקרת הרשאות בצד השרת בלבד.

11. פתרון טכנולוגי נבחר

המערכת היא מערכת וובית המורכבת מצד לקוח (Client), צד שרת (Server) ובסיס נתונים (DB), הנפרסת באמצעות תהליך CI/CD. הטכנולוגיות שנבחרו:

רכיב	טכנולוגיה / שירות
צד לקוח (Front End)	React 19 + Vite (TypeScript)
צד שרת (Back End)	Express (Node.js, TypeScript) — REST API
בסיס נתונים (DB)	PostgreSQL 18 + Prisma ORM
אימות והרשאות (Auth)	JWT בעוגיית HttpOnly + bcrypt + RBAC
תשלומים (Payments)	Stripe — Payment Intents + Webhooks
לוגים וניטור	Pino → Loki → Grafana
עיצוב	TailwindCSS (תמיכת RTL)
תשתית והרצה	Docker Compose + Nginx (reverse proxy) + Cloudflare Tunnel

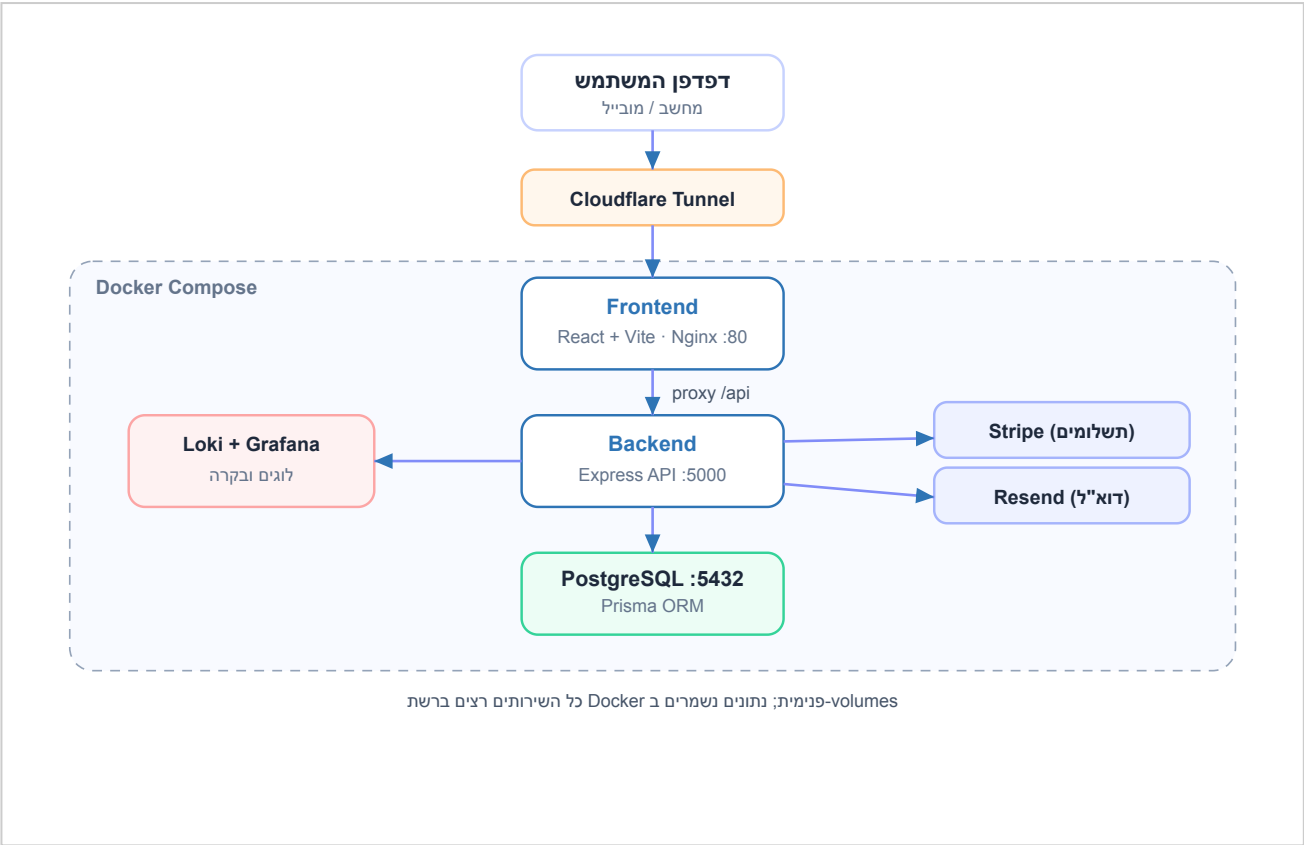
12. שפות הפיתוח

שפת **TypeScript** היא שפת הפיתוח המרכזית במערכת ומשמשת בכל הרכיבים — בצד הלקוח, בצד השרת ובאינטגרציות החיצוניות. היא משלבת את הפשטות של JavaScript עם מערכת טיפוסים סטטית (`strict: true`) המונעת שגיאות ומייעלת את העבודה.

- ב-**Front End** היא משמשת עם React ליצירת ממשקים אינטראקטיביים.
- ב-**Back End** היא משמשת עם Express לכתיבת לוגיקה עסקית ו-API.
- **SQL** — שפת בסיס הנתונים היחסי (PostgreSQL); נגזרת ומנוהלת אוטומטית על-ידי Prisma.

13. תיאור הארכיטקטורה הנבחרת

הארכיטקטורה שנבחרה מבוססת על הפרדה ברורה בין שלוש שכבות — **צד לקוח** (React), **צד שרת** (Express) ו**בסיס נתונים** (PostgreSQL) — כשכולן רצות כשירותי Docker נפרדים תחת Docker Compose. צד הלקוח מוגש על-ידי Nginx, המשמש גם כ-reverse-proxy ומעביר בקשות `/api` אל השרת. צד השרת עצמו בנוי **בשכבות** (Routes → Controllers → Services → Prisma), חסר-מצב, ומתקשר עם בסיס הנתונים דרך Prisma ORM. הארכיטקטורה תומכת בפיתוח מהיר, באבטחה מובנית וביכולת התרחבות.



תרשים 1.1 — ארכיטקטורת המערכת הכללית

14. חלוקה לתכניות ומודולים

תיאור	רכיב	צד המערכת
הגדרת מסלולי ה-API.	Routes	צד שרת
קישור בקשה/תשובה וקריאה ל-Services.	Controllers	
לוגיקה עסקית ושאליות Prisma מול בסיס הנתונים.	Services	
ייצוג טבלאות הנתונים (סטודיו, סניף, חדר, חוג, הרשמה, תשלום, משתמשים).	Prisma Schema (Models)	
אימות (JWT), הרשאות (RBAC), ולידציה (Zod) וטיפול בשגיאות.	Middleware	
התחברות, לוח בקרה, מערכת שעות, תלמידים, תשלומים, ניהול, נוכחות, עיון בחוגים.	מסכים (Components) לפי (תפקיד)	צד לקוח
רכיבי תצוגה גנריים (מודאל, כרטיס, סרגל, רשת לוז).	קומפוננטות משותפות (/common)	
ריכוז כל קריאות ה-HTTP מול השרת.	שכבת שירותים (services/api.ts)	

15. השרת

השרת מבוסס Node.js (Express) וכתוב ב-TypeScript. הוא רץ בקונטיינר node:20-alpine תחת Docker Compose, לצד קונטיינר postgres:18 וקונטיינר Nginx המגיש את צד הלקוח ומשמש כ-reverse-proxy לבקשות /api. גישה חיצונית מאובטחת מתאפשרת דרך Cloudflare Tunnel. השרת חסר-מצב (stateless) ומתקשר עם בסיס הנתונים באמצעות Prisma ORM; כל שירות מוגדר עם מדיניות הפעלה מחדש אוטומטית בעת קריסה.

16. ממשק המשתמש / GUI

המשתמש ניגש למערכת דרך הדפדפן (Chrome / Edge / Safari) ללא צורך בהתקנת אפליקציה, כאפליקציית עמוד יחיד (SPA) שנבנתה ב-React. הממשק עברי, בכיווניות RTL, ורספונסיבי — סרגל צד במחשב וניווט תחתון במובייל. האימות מתבצע מול השרת באמצעות עוגיית JWT.

17. ממשקים למערכות אחרות / API

המערכת מתממשקת לשירותים חיצוניים באמצעות API:

- **Stripe** — סליקת תשלומים מאובטחת ויצירת כוונות תשלום; אירועי Webhook מאומת-חתימה (`/api/webhooks/stripe`).
- **Resend / Nodemailer** — שליחת מיילים (הזמנות, איפוס סיסמה).
- **Loki + Grafana** — איסוף לוגים מובנים והצגתם בלוח בקרה.
- **Supabase** — מאותחל בצד לקוח, שמור לאחסון קבצים עתידי.

המערכת בנויה במבנה מודולרי המאפשר הוספת ממשקים נוספים בעתיד.

18. שימוש בחבילות תוכנה

צד	חבילה	תיאור / ייעוד
שרת	<code>express</code>	מסגרת שרת ובניית REST API.
	<code>@prisma/client</code>	ORM ושאלות בטוחות-טיפוסים מול PostgreSQL.
	<code>jsonwebtoken</code> · <code>bcryptjs</code>	אימות JWT והצפנת סיסמאות.
	<code>stripe</code>	אינטגרציית סליקה ו-Webhooks.
	<code>zod</code> · <code>sanitize-html</code>	ולידציה וניקוי קלט.
	<code>helmet</code> · <code>cors</code> · <code>express-rate-limit</code> · <code>pino</code>	אבטחה, בקרת גישה, הגבלת קצב ולוגים.
לקוח	<code>react</code> / <code>react-dom</code>	ספריית UI מבוססת קומפוננטות.
	<code>axios</code>	קריאות API מול השרת.
	<code>@stripe/react-stripe-js</code>	איסוף פרטי כרטיס מאובטח (Stripe Elements).
	<code>recharts</code> · <code>date-fns</code>	גרפים בלוח הבקרה וטיפול בתאריכים.
	<code>tailwindcss</code> · <code>lucide-react</code> · <code>vitest</code>	עיצוב, אייקונים ובדיקות יחידה.

19. שימוש במבני נתונים וארגון קבצים

בסיס הנתונים **PostgreSQL** משמש כמאגר המרכזי, עם טבלאות המייצגות ישויות: משתמשים, סטודיואים, סניפים, חדרים, קטגוריות, חוגים, הרשמות, נוכחות, תשלומים ועמלות. הקשרים הלוגיים: `studio → branch → room → class → enrollment → attendance` (בעל תפקיד) וכן `user` (בעל תפקיד).

ו-`payment`. כל המפתחות הראשיים הם `UUID`, חותמות הזמן `timestampz`, וכל טבלה תחומה ב-`studio_id`. קבצי המערכת מאורגנים לפי שכבות — `routes, controllers, services, middleware, schemas` בצד השרת, וקומפוננטות לפי תפקיד + שכבת `services` בצד הלקוח. המבנה המלא מוצג בפרק תרשים ERD.

20. תרשימי מערכת מרכזיים

- **Use Case** — מקרי שימוש עיקריים לכל סוג משתמש (מנהל-על, מנהל, מורה, תלמיד).
- **Sequence Diagram** — רצף הפעולות בין לקוח, שרת ובסיס נתונים (לדוגמה: הרשמה לחוג ותשלום).
- **Data Flow** — זרימת הנתונים בין רכיבי המערכת.

תרשימים מפורטים מוצגים בפרק "תהליכים מרכזיים במערכת". מקרי השימוש העיקריים:

#	תפקיד	תרחיש
UC-1	מנהל סטודיו	יוצר סניף, חדר וקטגוריה לסטודיו שלו.
UC-2	מנהל סטודיו	יוצר חוג חדש ומשייך לו מדריך, חדר, יום ושעה.
UC-3	מנהל סטודיו	מזמין מדריך/תלמיד באמצעות קישור הזמנה (JWT).
UC-4	מנהל סטודיו	רושם תלמיד לחוג ומנהל תשלומים.
UC-5	מדריך	צופה בלוח השיעורים שלו ומסמן נוכחות תלמידים.
UC-6	תלמיד	מעיין בחוגים, נרשם ומשלם תשלום מקוון.
UC-7	משתמש	מתחבר/מתנתק; מאפס סיסמה דרך קישור במייל.
UC-8	מנהל-על	צופה ומנהל את הפלטפורמה חוצה-הסטודיואים.

21. תיאור המרכיב האלגוריתמי-חישובי

המרכיב האלגוריתמי-חישובי המרכזי במערכת הוא **תהליך ההרשמה לחוג והתשלום**, לצד חישוב עמלות המדריכים ובקרת ההרשאות. בעת בקשת הרשמה מבצעת המערכת רצף בדיקות מסודר:

1. אימות קיום החוג ושיוכו לסטודיו הנכון (בידוד רב-דיירי).
2. בדיקת קיבולת — `current_enrollment < max_capacity`.
3. מניעת הרשמה כפולה — חיפוש הרשמה פעילה קיימת לאותו תלמיד וחוג.
4. זיהוי חוג חינם (`price = 0`) והפעלה מיידית; אחרת יצירת הרשמה בסטטוס `PENDING` והפניה לתשלום.

עיבוד תשלום אידמפוטנטי: בעת קבלת Webhook מ-Stripe, מתבצע עדכון מותנה בתוך טרנזקציה (`updateMany ... where status = 'PENDING'`) המבטיח שכל אירוע יעובד פעם אחת בלבד, גם אם

נשלח שוב. **חישוב עמלות מדריך** נגזר מהתשלום ונרשם בטבלת `instructor_commissions`. **בקרת הרשאות (RBAC)** וסינון לפי `studio_id` מבטיחים שכל משתמש רואה ופועל רק על הנתונים המורשים לו.

22. אבטחת מידע

המערכת משלבת מנגנוני אבטחה ברמת המשתמש, השרת והנתונים:

- אימות באמצעות **JWT בעוגיית HttpOnly** (שאינה נגישה ל-JavaScript) עם תפוגה.
- **הצפנת סיסמאות** באמצעות `bcrypt`.
- **הרשאות** (`requireRole`) (**RBAC**) המגבילות פעולות לפי תפקיד; כל הבקשות עוברות אימות ובקרה בצד השרת בלבד.
- **ולידציה וניקוי קלט** (`Zod + sanitize-html`) למניעת XSS והזרקות.
- **Helmet** (כותרות אבטחה/CSP), מדיניות **CORS** מבוקרת **והגבלת קצב** (`rate limiting`).
- **בידוד רב-דיירי** לפי `studio_id` בכל שאילתה.
- תקשורת מוצפנת **HTTPS** (דרך Cloudflare Tunnel) ולוגים מובנים לפעולות קריטיות (`Pino → Loki`) לצורך ביקורת וניטור.

23. משאבים הנדרשים לפרויקט

23.1 שעות עבודה וחלוקת עבודה

הפרויקט מתומחר בכ-350 שעות עבודה. חלוקה:

משימה	אחראים
תכנון מסד נתונים וארכיטקטורה	אופיר
פיתוח צד שרת (<code>Express + Prisma</code>)	אופיר
אינטגרציה לשירותים חיצוניים (<code>Stripe</code> , מיילים)	אופיר
פיתוח צד לקוח (<code>React</code>)	איתי
עיצוב UI/UX	איתי
בדיקות מערכת והרצה	איתי ואופיר

23.2 ציוד נדרש

- עמדות מחשב עם `macOS / Windows / Linux` וחיבור אינטרנט.
- מכשירי מובייל לבדיקת רספונסיביות.

23.3 תוכנות נדרשות

תוכנה	ייעוד
Visual Studio Code	עורך קוד
Git + GitHub	בקרת גרסאות ואחסון
Node.js + npm	סביבת ריצה וניהול חבילות
Docker Desktop	קונטיינריזציה והרצת השירותים
PostgreSQL / Prisma Studio	בסיס הנתונים וממשק ניהול
Stripe CLI	בדיקת Webhooks של תשלומים מקומית
Google Chrome	דפדפן וכלי פיתוח

23.4 ידע חדש נדרש

- עבודה עם Prisma ORM ומידול נתונים יחסי בטוח-טיפוסים.
- אינטגרציית תשלומים עם Stripe (Payment Intents, Elements, Webhooks).
- קונטיינריזציה ותזמור באמצעות Docker Compose.
- לוגים מובנים ובקרה עם Pino, Loki ו-Grafana.
- תכנון רב-דיירות (multi-tenancy) ובידוד נתונים, ואבטחה מבוססת JWT/RBAC.

23.5 ספרות ומקורות מידע

תיעוד רשמי של React, Vite, Express, Prisma, PostgreSQL, Stripe, Docker, TailwindCSS, JWT ו-Zod — ראו פרק ביבליוגרפיה.

24. תוכנית עבודה ושלבים למימוש הפרויקט

שלב	תיאור משימה	הערכת זמן (שעות)
שלב 1: תכנון והגדרות		
1.1	הקמת ניהול משימות והגדרת זרימת עבודה	5
1.2	תכנון בסיס הנתונים (סכמת Prisma)	20
1.3	לימוד והעמקה בטכנולוגיות (Prisma, Stripe, Docker)	30
1.4	תכנון ארכיטקטורה וזרימת נתונים	20
שלב 2: תשתית ואבטחה		
2.1	מנגנון אימות והרשאות (JWT, RBAC) ובידוד רב-דיירי	25
שלב 3: עיצוב וחווית משתמש (UI/UX)		
3.1	שרטוט ועיצוב מסכי המערכת (RTL)	15
3.2	בניית מערכת עיצוב וקומפוננטות משותפות	10
שלב 4: פיתוח ליבה		
4.1	ניהול סטודיו / סניפים / חדרים / חוגים (CRUD)	30
4.2	מערכת שעות וניהול תלמידים	30
4.3	הרשמה לחוג ולוגיקת קיבולת / כפילות	25
4.4	אינטגרציית תשלומים (Stripe + Webhooks)	30
שלב 5: דפי משתמשים וניהול		
5.1	לוחות בקרה לפי תפקיד + גרפים	25
5.2	ממשק מדריך וסימון נוכחות	20
5.3	ממשק מנהל-על (חוצה-סטודיואים)	15
שלב 6: בדיקות והטמעה		
6.1	בדיקות יחידה ואינטגרציה (Jest / Vitest)	15
6.2	בדיקות קצה ושיפורים	10
6.3	קונטיינריזציה והעלאה (Docker / CI-CD)	20

שלב	תיאור משימה	הערכת זמן (שעות)
סה"כ שעות עבודה משוער		כ-345 שעות

25. תכנון הבדיקות שיבוצעו

25.1 בדיקות תהליכיות (Full-Flow)

בדיקה תהליכית	תיאור הבדיקה
הרשמה והתחברות למערכת	בדיקת תהליך ההרשמה/התחברות המלא וניתוב לפי תפקיד.
התחברות עם פרטים שגויים	טיפול נכון בשגיאת אימות (שם משתמש/סיסמה לא נכונים).
עיון בחוגים והרשמה	תלמיד מעיין בקטלוג, נרשם לחוג ספציפי ומשלם.
תשלום מאובטח ואישור הרשמה	תשלום דרך Stripe ואישור אוטומטי של ההרשמה בעקבות Webhook.
דיווח נוכחות ע"י מורה	מורה מסמן נוכחות למפגש והנתונים נשמרים ומוצגים.
הזמנת משתמש ע"י מנהל	קישור ההזמנה מקצה תפקיד וסטודיו נכונים.
הפקת מדדים בלוח הבקרה	הצגה נכונה של נתונים סטטיסטיים (הכנסות, הרשמות, נוכחות).

25.2 בדיקות יחידה (Unit Tests)

תהליך / מודול	בדיקות אימות נתונים / תקינות
authController	שדות חובה, אימות פורמט מייל וסיסמה, התחברות תקינה/שגויה.
יצירת חוג	שדות חובה ותקינות זמנים.
enrollmentService	בדיקת קיבולת, מניעת כפילות רשומה, חוג חנים.
paymentService	תקינות שדות ואידמפוטנטיות (מניעת עיבוד/חיוב כפול).
בידוד רב-דיירי	ודאות שכל שאילתה מסוננת לפי studio_id.

בדיקות צד שרת רצות תחת Jest + Supertest; בדיקות צד לקוח תחת Vitest + React Testing Library.

26. בקרת גרסאות (Version Control)

ניהול הגרסאות מתבצע באמצעות Git עם אחסון ב-GitHub. השימוש ב-Git מאפשר עבודה שיתופית נוחה, מעקב אחר שינויים, יצירת ענפים (branches) לפיצ'רים שונים ומניעת התנגשויות בקוד. נעשה שימוש

ב- `feat:`, `fix:`, `refactor:` *Conventional Commits* (... ובבדיקות אוטומטיות ב-pre-commit (Husky).

חתימת המנחה האישי

חתימת הסטודנט

נספחים: דיפלומת מנחה, תרשימי מערכת, מסכי מערכת לדוגמה ועוד.

2. הצהרת הסטודנטים

אנו, החתומים מטה, מצהירים בזאת כי פרויקט הגמר "Classly — מערכת לניהול סטודיו" נעשה על-ידינו באופן עצמאי, וכי כל המקורות והכלים שבהם נעזרנו מצוינים במפורש בגוף העבודה ובביבליוגרפיה.

אנו מצהירים כי הקוד נכתב על-ידינו, וכי במקומות בהם נעזרנו בספריות צד-שלישי (כמפורט בפרק חבילות התוכנה) — הדבר נעשה בהתאם לרישיונות השימוש שלהן. כל שימוש בקוד או ברעיון של אחר צוין כנדרש.

הבנו את חשיבות היושרה האקדמית, ואנו אחראים לתוכן העבודה במלואה.

חתימת הסטודנט א'

חתימת הסטודנט ב'

חתימת המנחה האישי

3. הוקרות

אנו רוצים להוקיר תודה **למנחה המקצועי שלנו**, שליווה אותנו לאורך כל הדרך ועזר לנו עם כל שאלה מקצועית, ארכיטקטונית ואלגוריתמית, ונענה לכל בקשה באדיבות ובמסירות.

כמו כן, נרצה להודות **למכללה** על הסביבה הלימודית, על החומרים וההדרכה שסיפקה לנו, ועל המשוב לאורך הפרויקט שעזר לנו להגיע לתוצר הסופי.

תודה מיוחדת לכל מי שתרו ממצמו ומניסיונו, ולמשפחותינו על התמיכה לכל אורך הפיתוח.

4. תקציר

Classly היא מערכת ווב מקצה-לקצה (full-stack) לניהול סטודיו כושר ובריאות, הבנויה כפלטפורמת **SaaS רב-דיירית**: כל סטודיו הוא דייר מבודד עם הסניפים, החדרים, החוגים, התלמידים והתשלומים שלו.

המערכת תומכת בארבעה תפקידים — **מנהל-על, מנהל סטודיו, מדריך ותלמיד** — וכל תפקיד רואה ממשק מותאם. מנהל הסטודיו מנהל סניפים, חדרים, חוגים, מערכת שעות, תלמידים ותשלומים; המדריך צופה בלוח שלו ומסמן נוכחות; התלמיד מעיין בחוגים, נרשם ומשלם תשלום מקוון מאובטח דרך Stripe.

צד השרת נכתב ב-**Express** בארכיטקטורת שכבות (Routes → Controllers → Services → Prisma), עם אימות מבוסס **JWT בעוגיית HttpOnly** ואכיפת הרשאות לפי תפקיד. צד הלקוח נכתב ב-**React**, בעברית RTL ומותאם למובייל. כל המערכת רצה תחת **Docker Compose**, עם בסיס נתונים **PostgreSQL** וניהול גישה דרך **Prisma ORM**. לוגים מובנים נאספים ב-Loki ומוצגים ב-Grafana.

ערכי הליבה של המערכת: **בידוד נתונים מלא** בין סטודיואים, **אבטחה כברירת מחדל**, **קוד מודולרי וגמיש**, ו**חוויית משתמש פשוטה ונגישה**.

5. עקרונות בסיסיים בפרויקט

העקרונות שהנחו אותנו בכתיבת הקוד ובמימוש המערכת

קוד נקי, מסודר וקריא + Code Review

הקפדנו על כתיבת קוד קריא ועקבי. בכל פעם שאחד מאיתנו סיים משימה, השני עבר על הקוד ובחן אותו (code review) על מנת לשמור על איכות ועל קריאות.

ארכיטקטורת שכבות והפרדת אחריות

צד השרת בנוי בשכבות עם תפקיד ברור לכל אחת (Routes / Controllers / Services / Prisma). הפרדה זו מפשטת את הפיתוח, מקלה על איתור באגים, והופכת את המערכת לגמישה יותר לשינויים.

הפרדת צד לקוח וצד שרת + TypeScript Strict

הפרדנו את הצדדים כך שכל אחד עומד בפני עצמו. בשני הצדדים נעשה שימוש ב-TypeScript במצב strict, המעלה את בטיחות הקוד ומפחית שגיאות זמן-ריצה.

רב-דיירות ובידוד נתונים

כל שאילתה תחומה לפי `studio_id`, כך שאף משתמש אינו יכול לגשת לנתונים של סטודיו אחר. זהו עיקרון מנחה בכל שכבת השירותים.

מניעת חזרה על קוד — קומפוננטות גנריות

בנינו קומפוננטות מוצגות גנריות ומשותפות (תיקיית `components/common/`: `BaseModal`, `StatCard`, `ClassCard`, `FormFields`, `ScheduleGrid`) המשמשות שוב ושוב — כך נמנעת חזרה על קוד והמערכת קלה להרחבה.

שכבת שירותים אחת בצד לקוח

כל קריאות ה-HTTP עוברות דרך `services/api.ts` (מופע `axios` יחיד). אף קומפוננטה אינה קוראת ל-`axios` ישירות — מה שמרכז את הטיפול בעוגיות, בשגיאות וב-`interceptors`.

אבטחה כברירת מחדל

JWT בעוגיית `HttpOnly`, הצפנת סיסמאות עם `bcrypt`, כותרות אבטחה עם `Helmet/CSP`, ניקוי קלט (`sanitize-html`) וולידציה עם `Zod`, והגבלת קצב בקשות.

לוגים מובנים ובקרה

כל בקשה מקבלת מזהה (`requestId`) ולוגר ייעודי (Pino). הלוגים נשלחים ל-Loki ומוצגים ב-Grafana, כך שניתן לעקוב אחר תקלות ולנתח ביצועים.

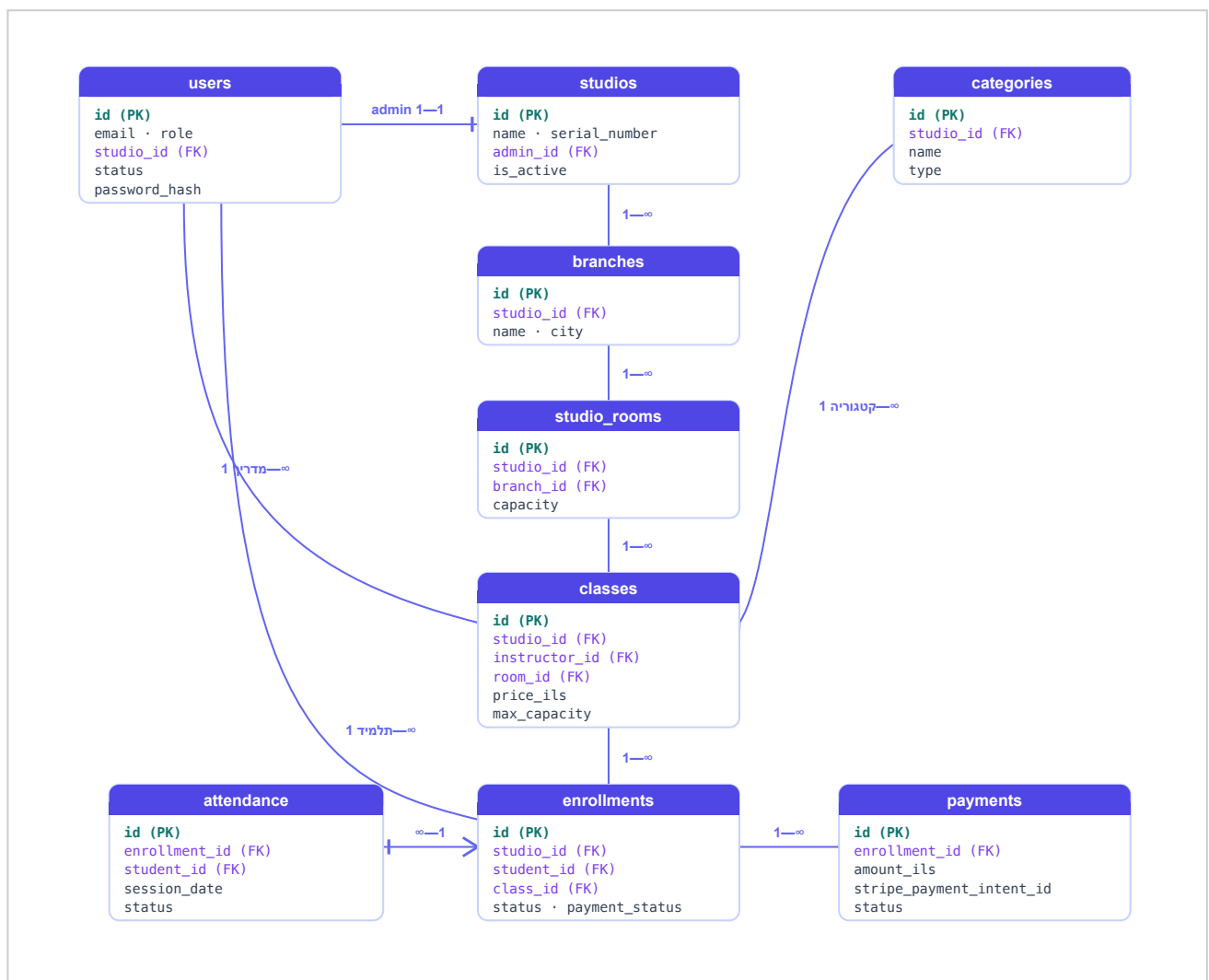
Containerization + Conventional Commits

כל המערכת רצה תחת Docker Compose לסביבה אחידה וניתנת לשחזור. ניהול הקוד ב-Git עם Conventional Commits ובדיקות אוטומטיות ב-pre-commit (Husky: Prettier + typecheck) + Husky: Prettier + typecheck). (בדיקות).

6. תרשים ERD

מודל הנתונים היחסי (Entity–Relationship Diagram)

המודל בנוי סביב היררכיית הבעלות studios → branches → studio_rooms → classes → enrollments → attendance. studios היא הדייר (tenant), וכל שאר הטבלאות התפעוליות נושאות שדה studio_id לבידוד. מחיקת סטודיו מפילה בשרשור (cascade) את כל נתוניו. כל המפתחות הראשיים הם UUID.



תרשים 6.1 — מודל הנתונים (ישויות מרכזיות, מפתחות וקשרים). PK = מפתח ראשי · FK = מפתח זר · ∞ = רבים

ישויות עזר נוספות

בנוסף לישויות המרכזיות שבתרשים, הסכמה כוללת טבלאות תומכות (כולן תחומות ב-studio_id או מקושרות ל-users):

טבלה	תפקיד
<code>instructor_commissions</code>	עמלות מדריך לפי חוג (אחוז/קבוע, סטטוס תשלום).
<code>schedule_sessions</code>	מופעי מפגש קונקרטיים של חוג בתאריך נתון.
<code>notifications</code>	התראות למשתמש (נקרא/לא נקרא).
<code>audit_logs</code>	תיעוד פעולות (actor, action, changes JSONB).
<code>pending_registrations</code>	הרשמות ממתונות + טוקן הזמנה, פגות תוקף.
<code>password_reset_tokens</code>	טוקני איפוס סיסמה (hash, פג-תוקף, used).

הערה — אינדקסים

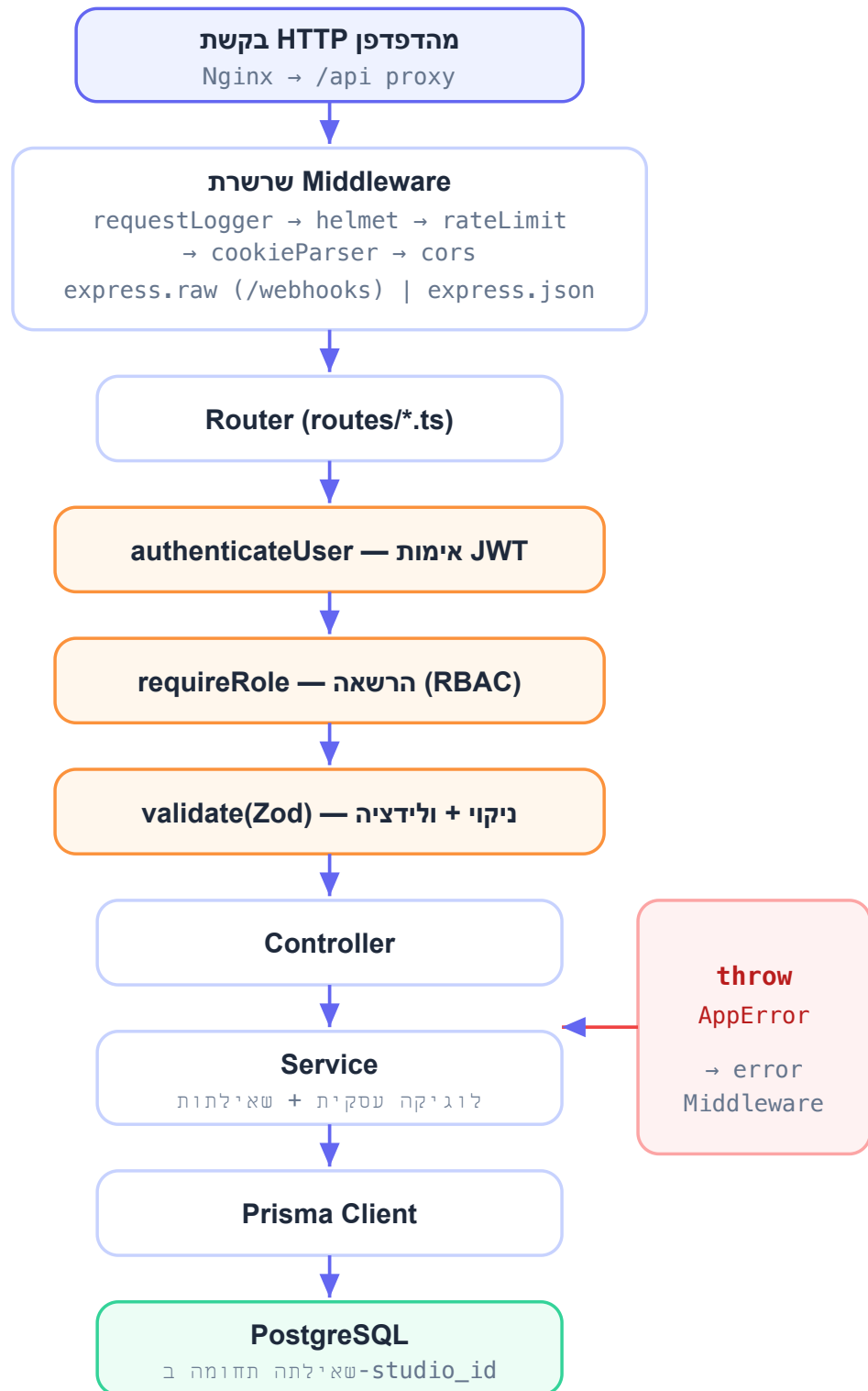
קיימים אינדקסים ייחודיים על `users.email`, `studios.serial_number`, ועל הצמד `payments.stripe_payment_intent_id` (enrollment_id, session_date) ב-attendance (מניעת כפילות סימון נוכחות לאותו מפגש).

7. צד שרת

Express + TypeScript בארכיטקטורת שכבות, מול PostgreSQL דרך Prisma

צד השרת נכתב ב-**Express** וחושף REST API תחת הקידומת `/api`. הוא בנוי בשכבות, כאשר כל בקשה עוברת מסלול קבוע: שרשרת middleware (לוגים, אבטחה, אימות, הרשאה, ולידציה) ואז *Controller* → *Service* → *Prisma* בסיס הנתונים.

7.1 ארכיטקטורת השכבות וזרימת הבקשה



תרשים 7.1 — מסלול הבקשה דרך שכבות השרת

השכבות ותפקידיהן:

- **Routes** — מגדירות אילו מסלולי API קיימים ואילו middleware רצים על כל אחד.
- **Middleware** — אחראי על נושאים רחביים: לוגים, אבטחה, אימות זהות, הרשאה וולידציה.
- **Controller** — קורא את הבקשה (`req.body` , `req.params` , `req.studioId`), קורא לשירות, ומעצב את התשובה.
- **Service** — מכיל את הלוגיקה העסקית (קיבולת, תמחור, מעברי סטטוס) ואת שאילתות Prisma.
- **Prisma** — שכבת הגישה לבסיס הנתונים, ממירה קריאות type-safe ל-SQL.

7.2 יתרונות ארכיטקטורת השכבות

1. הפרדת אחריות

כל שכבה אחראית על תחום אחד בלבד, מה שמפחית תלות ומאפשר לכל שכבה להתמקד בתפקידה.

2. תחזוקה קלה

קל לתקן באגים — כל נושא מטופל בשכבה אחת, ולכן קל לבודד את מקור התקלה.

3. גמישות להרחבה

הוספת פיצ'ר או ישות חדשה אינה דורשת שינוי בשכבות אחרות; המערכת גמישה לשינויים.

4. בדיקות

ניתן לבדוק כל שכבה בנפרד (למשל בדיקת שירות מול Prisma מדומה).

קובץ ה-middleware המרכזי (app.ts)

סדר ה-middleware קריטי: `express.raw` עבור `/api/webhooks` חייב לרוץ לפני `express.json`, כדי ש-Stripe יוכל לאמת את חתימת הגוף הגולמי.

server/src/app.ts

```
export const app = express();
app.set("trust proxy", 1);

app.use(requestLogger); // מציג req.logger + req.requestId
app.use(helmet({ contentSecurityPolicy: { /* CSP directives */ } }));
app.use(rateLimit({ windowMs: 15 * 60 * 1000, max: 5000 }));
app.use(cookieParser()); // קריאת עוגיית classly_token
app.use(cors({ origin: /* whitelist */ allowedOrigins, credentials: true }));

// ⚠️ raw body ל-Webhooks – חייב לפני express.json
app.use("/api/webhooks", express.raw({ type: "application/json" }), webhookRoutes);
app.use(express.json()); // רגיל JSON – כל שאר המסלולים

app.use("/api/auth", authRoutes);
app.use("/api/courses", courseRoutes);
app.use("/api/enrollments", enrollmentRoutes);
app.use("/api/payments", paymentRoutes);
// ... students, instructors, attendance, dashboard, studios, branches, rooms

app.get("/api/health", (req, res) => res.json({ status: "OK" }));
app.use(errorHandler); // טיפול שגיאות מרוכז
```

תמונה 7.1 — מחסנית ה-middleware ורישום המסלולים (app.ts)

7.3 ה-API המרכזיים

כל המסלולים תחת `/api` מסלולים ציבוריים: `/api/auth/*`, `/api/health`, `/api/webhooks/stripe`. כל השאר דורשים אימות, ורובם גם הרשאת תפקיד.

קבוצת מסלולים	תפקיד
/api/auth/*	התחברות, הרשמה, me, התנתקות, שכחתי/איפוס סיסמה.
/api/courses/*	ניהול חוגים (יצירה, עדכון, רשימה, חוגי מדריך/זמינים).
/api/enrollments/*	הרשמה לחוג, הרשמות שלי, רשימת נרשמים, ביטול.
/api/payments/*	היסטוריית תשלומים, יצירת כוונת תשלום.
/api/students, /api/instructors	ניהול תלמידים ומדריכים.
/api/attendance/*	סימון וצפייה בנוכחות.
/api/dashboard/*	מדדים ללוח הבקרה (מנהל/מדריך).
/api/studios, /api/branches, /api/rooms	ניהול סטודיו, סניפים וחדרים.
/api/webhooks/stripe	קליטת אירועי Stripe (אימות חתימה, גוף גולמי).

7.4 הזדהות והרשאות

האימות מבוסס **JWT בעוגיית HttpOnly** בשם `classly_token` (JavaScript) אינו יכול לקרוא אותה — הגנה מפני גניבת טוקן ב-XSS). ה-`authenticateUser` middleware מאמת את הטוקן, שולף את המשתמש מה-DB, דוחה משתמש מושעה, ומצרף `req.user` ו-`req.studioId` לבקשה. `requireRole` אוסף הרשאות לפי תפקיד.

server/src/middleware/authMiddleware.ts

```
export const authenticateUser = async (req, res, next) => {
  const token = req.cookies?.classly_token;
  if (!token) return res.status(401).json({ error: "Not authenticated" });

  try {
    // 1. מקומית JWT-אימות חתימת ה
    const decoded = jwt.verify(token, environment.jwtSecret) as JwtPayload;

    // 2. שליפת פרופיל המשתמש מבסיס הנתונים
    const userData = await prisma.users.findUnique({
      where: { id: decoded.userId },
      select: { id: true, role: true, studio_id: true, status: true },
    });
    if (!userData) return res.status(403).json({ error: "User not found" });
    if (userData.status === "SUSPENDED")
      return res.status(403).json({ error: "Account is suspended" });

    // 3. צירוף המשתמש לבקשה עבור השכבות הבאות
    req.user = userData;
    req.studioId = userData.studio_id || undefined;
    next();
  } catch (err) {
    if (err.name === "TokenExpiredError")
      return res.status(401).json({ error: "Token expired" });
    return res.status(401).json({ error: "Invalid token" });
  }
};

// אכיפת תפקידים נדרשים על מסלול
export const requireRole = (allowedRoles: string[]) => (req, res, next) => {
  if (!req.user?.role || !allowedRoles.includes(req.user.role))
    return res.status(403).json({ error: "Insufficient permissions" });
  next();
};
```

תמונה 7.2 — אימות זהות ואכיפת הרשאות (authMiddleware.ts)

בעת התחברות מוצלחת נחתם הטוקן ונשמר בעוגייה עם ההגדרות הבאות (תוקף 7 ימים; `secure` מופעל רק בייצור):

server/src/controllers/authController.ts

```
const AUTH_COOKIE_OPTIONS = {
  httpOnly: true, // JS לא יכול לקרוא
  secure: process.env.NODE_ENV === "production", // HTTPS בלבד בייצור
  sameSite: "lax", // חלקית CSRF הגנת
  maxAge: 7 * 24 * 60 * 60 * 1000, // ימים 7
  path: "/",
};
// login: bcrypt.compare → jwt.sign({ userId, email, role }) → res.cookie(...)
res.cookie("classly_token", token, AUTH_COOKIE_OPTIONS);
```

תמונה 7.3 — הגדרת עוגיית האימות (authController.ts)

7.5 תפקידים (Roles)

תפקיד	הרשאות
SUPER_ADMIN	ניהול הפלטפורמה חוצה-הסטודיואים והגדרות מערכת.
ADMIN	ניהול מלא של הסטודיו שלו — סניפים, חדרים, חוגים, תלמידים, תשלומים, צוות והגדרות.
INSTRUCTOR	לוח שיעורים, רשימת תלמידים בחוגיו, סימון נוכחות.
STUDENT	עיון בחוגים, הרשמה ותשלום, צפייה בהרשמות שלו.
PARENT	קיים בסכמה (רשומות ילדים) — שמור לשימוש עתידי.

7.6 טיפול בשגיאות מרכז

בקרים ושירותים לעולם אינם כותבים `res.status(500)...` ישירות, אלא **זורקים** `AppError` שמטופל ב-middleware מרכזי. כך התשובה ללקוח אחידה, והשגיאה נרשמת ללוג.

server/src/utils/AppError.ts · middleware/errorMiddleware.ts

```
export class AppError extends Error {
  constructor(public message: string, public statusCode: number) {
    super(message);
    this.isOperational = true;
  }
}

// errorMiddleware – handler מרכזי
const errorHandler = (err, req, res, next) => {
  const statusCode = err.statusCode || 500;
  req.logger.error({ message: err.message, statusCode, path: req.path });

  // נחשפות ללקוח; שגיאות תכנות – הודעה גנרית (AppError) שגיאות מתוכננות
  const message = err.isOperational ? err.message : "Something went wrong.";
  res.status(statusCode).json({ status: "error", message });
};
```

תמונה 7.4 — מחלקת AppError ו-handler השגיאות המרכזי

7.7 ולידציה וניקוי קלט

ה- `validate(schema)` middleware מבצע שני שלבים לפני שהבקשה מגיעה ל-Controller: (1) ניקוי תגיות HTML מכל שדה מחרוזת (למעט שדות רגישים כמו סיסמאות וטוקנים), ו-(2) ולידציה מול סכמת **Zod**. הניקוי נמצא במקום אחד — כך שאף מסלול לא יכול "לשכוח" אותו.

server/src/middleware/validate.ts

```
const SENSITIVE_FIELDS = new Set(["password", "token", "invitationToken", "newPassword"]);

export function validate(schema: ZodSchema) {
  return (req, _res, next) => {
    req.body = sanitizeBody(req.body); // 1. הסרת תגיות HTML
    const result = schema.safeParse(req.body); // 2. ולידציית Zod
    if (!result.success) {
      const first = result.error.errors[0];
      return next(new ValidationError(first.message, first.path.join(".")));
    }
    req.body = result.data;
    next();
  };
}
```

תמונה 7.5 — middleware של ניקוי קלט ולידציית Zod

7.8 התממשקות מול בסיס הנתונים (Prisma)

הגישה לבסיס הנתונים נעשית דרך **Prisma**. כל ישות מוגדרת כ-model בסכמה, וממנה נגזרים גם הטיפוסים וגם ה-SQL. דוגמה למודל ההרשמות, עם המפתחות הזרים והקשרים:

server/prisma/schema.prisma

```
model enrollments {
  id          String @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
  studio_id   String @db.Uuid
  student_id  String @db.Uuid
  class_id    String @db.Uuid
  status      String? @default("ACTIVE") // ACTIVE | PENDING | CANCELLED
  payment_status String? @default("PENDING") // PENDING | PAID | OVERDUE

  studio studios @relation(fields: [studio_id], references: [id], onDelete: Cascade)
  student users @relation("EnrollmentStudent", fields: [student_id], references: [id])
  class classes @relation(fields: [class_id], references: [id], onDelete: Cascade)
  payments payments[]
  attendance attendance[]
}
```

תמונה 7.6 — מודל Prisma של ישות ההרשמות (schema.prisma)

בידוד רב-דיירי — studio_id בכל שאילתה

עיקרון מנחה: כל שאילתה תחומה ב-studio_id. במקום `findUnique({ where: { id } })` משתמשים ב-`findFirst({ where: { id, studio_id } })`, ומתייחסים לכישלון כ-404 — כך משתמש מסטודיו אחד לא יכול לגשת לנתוני סטודיו אחר גם אם ניחש מזהה.

server/src/services/*Service.ts

```
// שאילתה תחומה לדייר — מונעת דליפת נתונים בין סטודיואים
const record = await prisma.payments.findFirst({
  where: { id, studio_id: studioId }, // תמיד מסננים לפי הדייר
});
if (!record) throw new AppError("Not found", 404);
```

תמונה 7.7 — תבנית שאילתה תחומת-דייר

טרנזקציות — הרשמה + תשלום יחד

פעולה הנוגעת בכמה טבלאות עוטפת אותן ב-`prisma.$transaction`, כך שהן מתבצעות יחד (atomic). למשל, יצירת הרשמה ורישום שורת תשלום מתבצעים כיחידה אחת:

server/src/services/enrollmentService.ts

```
await prisma.$transaction(async (tx) => {  
  // 1. ולידציית קיבולת + מניעת הרשמה כפולה, ואז יצירת ההרשמה  
  const { enrollment } = await EnrollmentService.enrollStudent(  
    studioId, studentId, classId, "PENDING", "PENDING", undefined, tx,  
  );  
  // 2. באותה טרנזקציה – PENDING רישום שורת תשלום ראשונית במצב  
  await PaymentService.createPaymentRecord(  
    { studioId, studentId, enrollmentId: enrollment.id, amount, stripePaymentIntentId  
  }, tx,  
  );  
});
```

תמונה 7.8 — טרנזקציה אטומית של הרשמה + תשלום

7.9 אינטגרציית Stripe ו-Webhooks

תהליך התשלום: השרת יוצר **PaymentIntent** ומחזיר ללקוח `clientSecret`; הלקוח מאשר את התשלום מול Stripe; ולבסוף Stripe שולח **Webhook** מאומת-חתימה לשרת, אשר מעדכן את התשלום וההרשמה בתוך טרנזקציה.

server/src/services/paymentService.ts

```
// יצירת כוונת תשלום (הסכום באגורות)
static async createIntent(amount, currency = "ils", description?) {
  if (!amount || amount <= 0) throw new AppError("Invalid amount", 400);
  const paymentIntent = await stripe.paymentIntents.create({
    amount: Math.round(amount * 100),
    currency,
    automatic_payment_methods: { enabled: true },
  });
  return { clientSecret: paymentIntent.client_secret, id: paymentIntent.id };
}

// הגוף הגולמי Webhook-אימות חתימת ה
static constructEvent(payload: Buffer, signature: string) {
  return stripe.webhooks.constructEvent(payload, signature, webhookSecret);
}

// אידמפוטנטיות מפני שליחה חוזרת (status = PENDING) → עדכון מותנה
const result = await prisma.$transaction(async (tx) => {
  await tx.payments.updateMany({
    where: { stripe_payment_intent_id: paymentIntentId, status: "PENDING" },
    data: { status: "SUCCEEDED", paid_date: new Date() },
  });
  // הפעלת ההרשמה בעקבות התשלום
  await tx.enrollments.updateMany({
    where: { id: enrollmentId, payment_status: { not: "PAID" } },
    data: { payment_status: "PAID", status: "ACTIVE" },
  });
});
```

תמונה 7.9 — יצירת כוונת תשלום, אימות Webhook ועדכון אידמפוטנטי (paymentService.ts)

למה גוף גולמי?

אימות חתימת ה-Webhook של Stripe מתבצע על ה-bytes המקוריים. לכן `express.raw` חייב לרוץ לפני `express.json` עבור `/api/webhooks` — אחרת החתימה לא תתאמת.

7.10 לוגים ובקרה (Pino + Loki)

כל בקשה מקבלת `requestId` ולוגר-ילד ייעודי (`req.logger`) המצרף הקשר (כמו `studioId`, `enrollmentId`). הלוגים מובנים כ-JSON, נשלחים ב-transport של `pino-loki` אל `Loki`, ומוצגים ב-Grafana. לעולם אין רושמים סיסמאות, טוקנים או גוף בקשה מלא.

server/src/services/paymentService.ts

```
const serviceLogger = logger.child({ service: "PaymentService", method: "createIntent"
});
serviceLogger.info({ amount, currency }, "Starting createIntent");
// ... בתוך handler: req.logger.info({ studioId, enrollmentId }, "Enrollment
created");
```

תמונה 7.10 — לוגר מובנה עם הקשר (Pino child logger)

8. צד לקוח

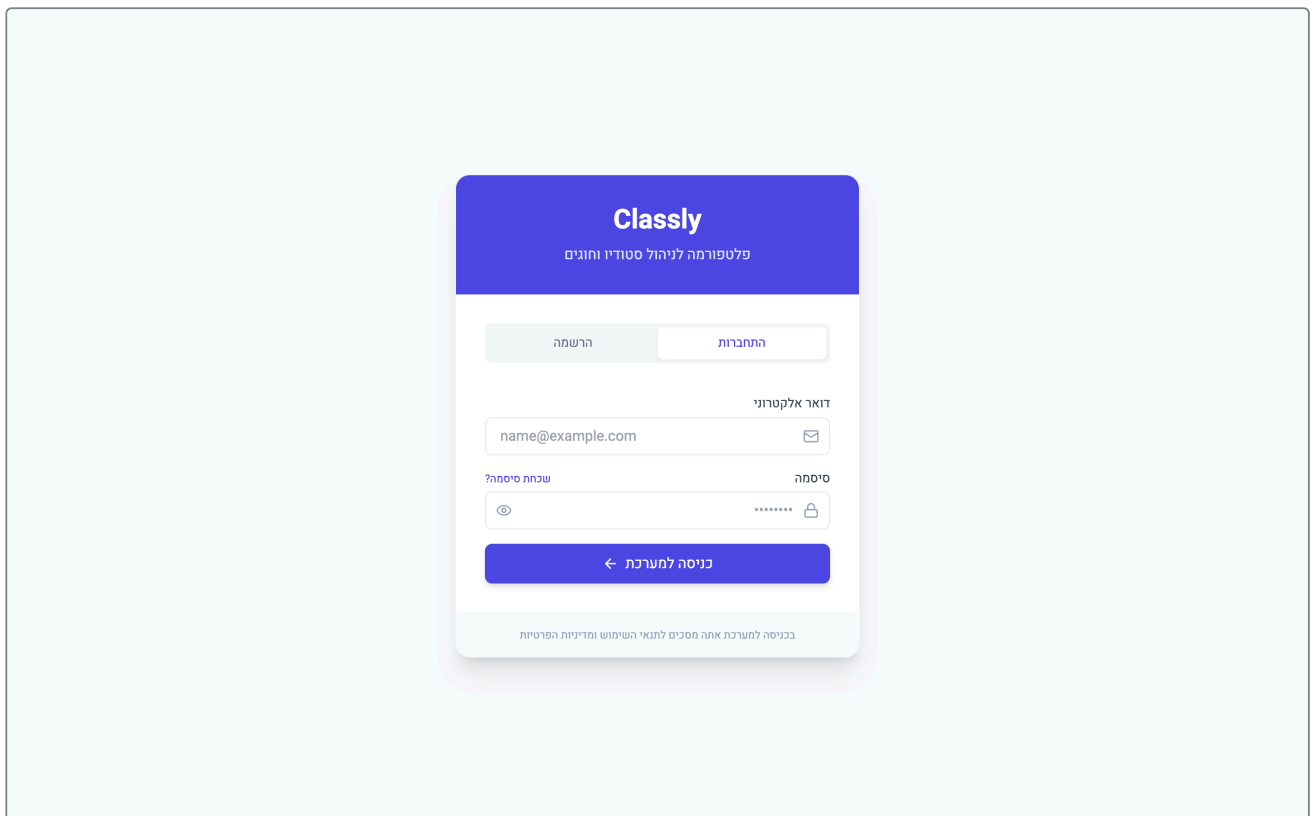
React 19 + Vite + TypeScript · עברית RTL · מותאם למובייל

צד הלקוח הוא אפליקציית עמוד יחיד (SPA) ב-React. הקומפוננטות מאורגנות לפי תפקיד, כל קריאות ה-HTTP עוברות דרך שכבת שירותים אחת (`api.ts`), ומצב ההזדהות מנוהל ב- `App.tsx` . כל הצילומים שלהלן נלקחו מהמערכת החיה.

8.1 מסכים מרכזיים במערכת

מסך ההתחברות והרשמה

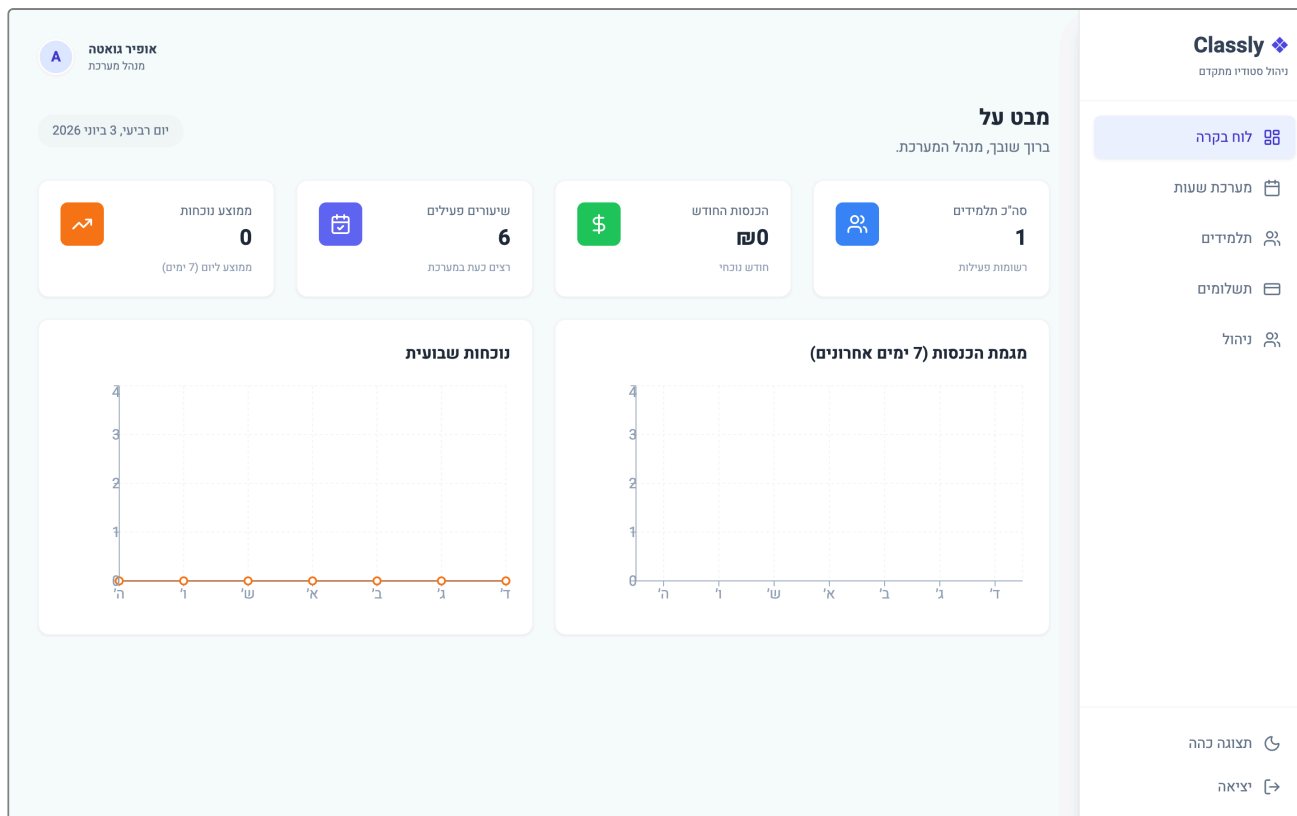
נקודת הכניסה למערכת. ניתן לעבור בין "התחברות" ל"הרשמה", לאפס סיסמה, ולהירשם דרך קישור הזמנה. הטוקן נשמר בעוגיית HttpOnly, והמשתמש מנותב למסך המתאים לתפקידו.



תמונה 8.1 — מסך ההתחברות

לוח בקרה — מנהל סטודיו

לוח הבקרה של המנהל מציג מדדים מרכזיים (ממוצע נוכחות, שיעורים פעילים, הכנסות החודש, סך תלמידים) וגרפים (נוכחות שבועית ומגמת הכנסות) באמצעות ספריית `recharts` .



תמונה 8.2 — לוח הבקרה של מנהל הסטודיו

מערכת שעות

תצוגת לוח השיעורים השבועי וניהול חוגים. לחיצה על "שיבוץ שיעור חדש" פותחת מודל ליצירת חוג (ראו תמונה 8.16).

A

אופיר גואטה

מנהל מערכת

ניהול סטודיו מתקדם

לוח בקרה

מערכת שעות

תלמידים

תשלומים

ניהול

תצוגה כהה

יציאה

שיבוץ שיעור חדש

מערכת שעות

ניהול השיעורים השבועי שלך

סניף ראשי

סניף סמילנסקי

סניף פולג

ראשון

שני

שלישי

רביעי

חמישי

שישי

שבת

כל הרמות

כל המדריכים

סיוון

2 אולם חדרות

08:00

09:00

10:00

11:00

ניהול תלמידים

אופיר גואטה
מנהל מערכת

+ הוסף תלמיד

ייצא CSV

ניהול תלמידים

ניהול הרשמות ופרטי תלמידים (1 רשומות)

כל השיעורים

סינון לפי שיעור:

חיפוש לפי שם או אימייל...

תלמיד	שיעור	סטטוס	פרטי קשר	הצטרף	פעולות
1 יעלה סטרון זדהה: 42bcad	בלט, מודרני, היפ הופ, יוגה למתחילים	פעיל	student1@student1.com 0542473647	8.5.2026	:

< הבא

עמוד 1 מתוך 1

> הקודם

Classly

ניהול סטודיו מתקדם

לוח בקרה

מערכת שעות

תלמידים

תשלומים

ניהול

תצוגה כהה

יצאה

תשלומים וחשבוניות

A

אופיר גואטה

מנהל מערכת

+

תשלום חדש

תשלומים וחשבוניות

נהל את התשלומים והצג היסטוריית חיובים

\$

יתרה לתשלום

₪0.00

סה"כ שולם (הצטברות)

₪0

ייצוא

חפש לפי מספר חשבונית...

תאריך	מספר חשבונית	שיטת תשלום	סכום	סטטוס	קבלה
8.5.2026	-	אשראי	₪120	ממתין	הורד PDF
8.5.2026	-	אשראי	₪80	ממתין	הורד PDF

ניהול סטודיו מתקדם

לוח בקרה

מערכת שעות

תלמידים

תשלומים

ניהול

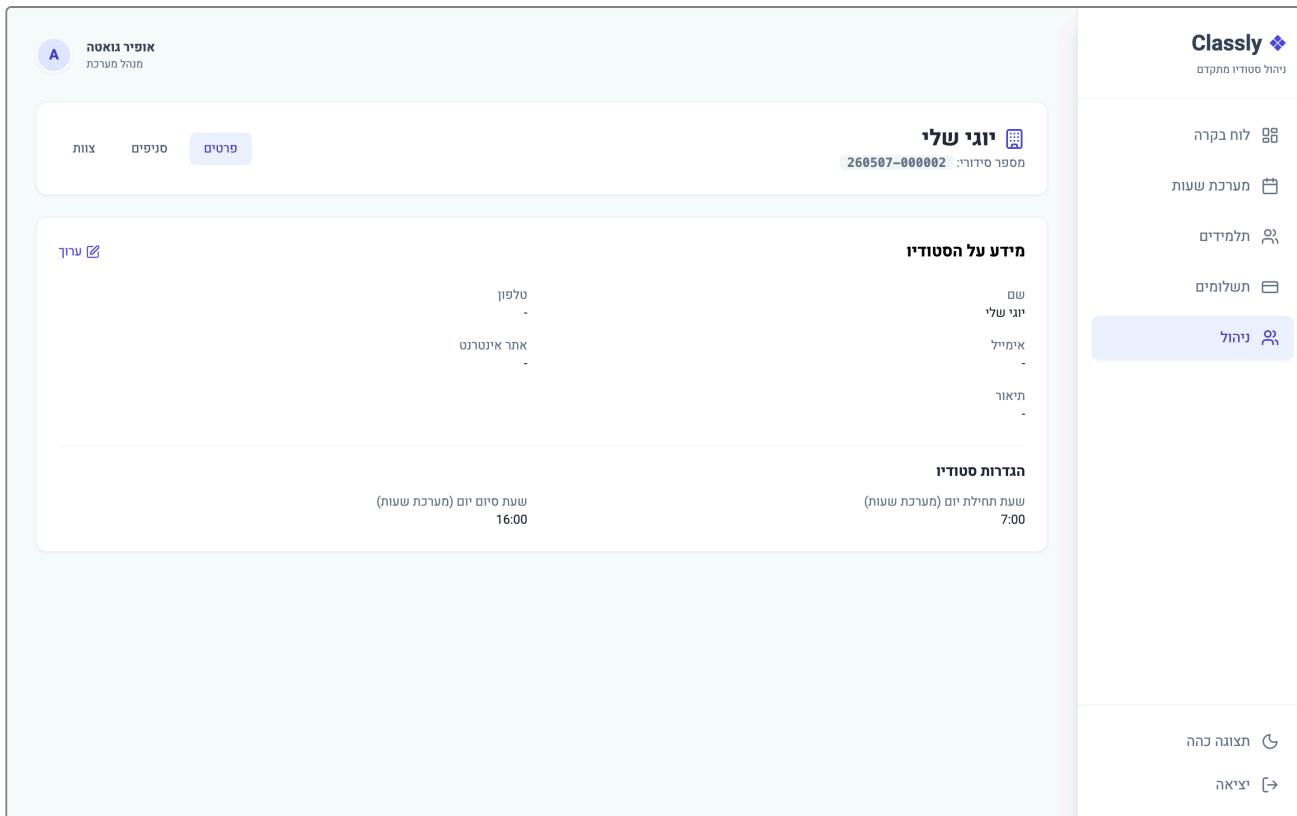
תצוגה כהה

יציאה

תמונה 8.5 — מסך התשלומים והחשבוניות

ניהול הסטודיו

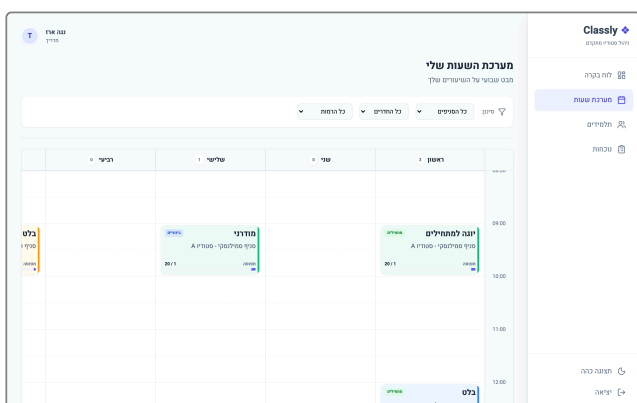
מסך הניהול מרכז את פרטי הסטודיו, הסניפים והצוות (מדריכים), כולל הזמנת אנשי צוות חדשים.



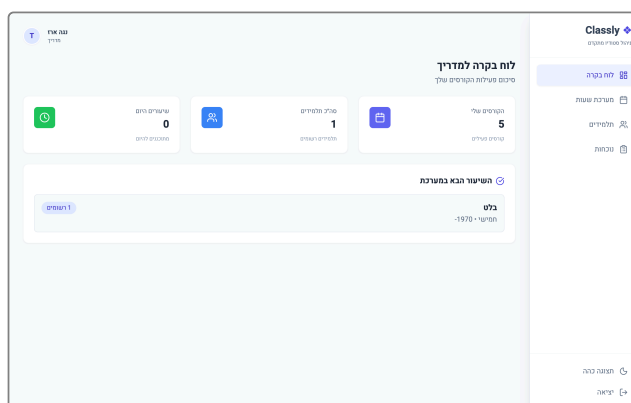
תמונה 8.6 — מסך ניהול הסטודיו

מסכי המדריך

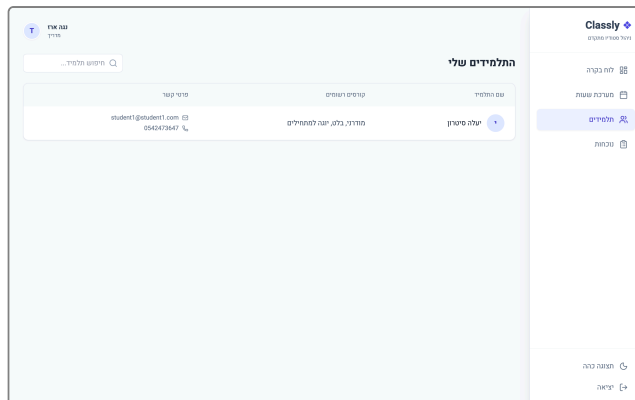
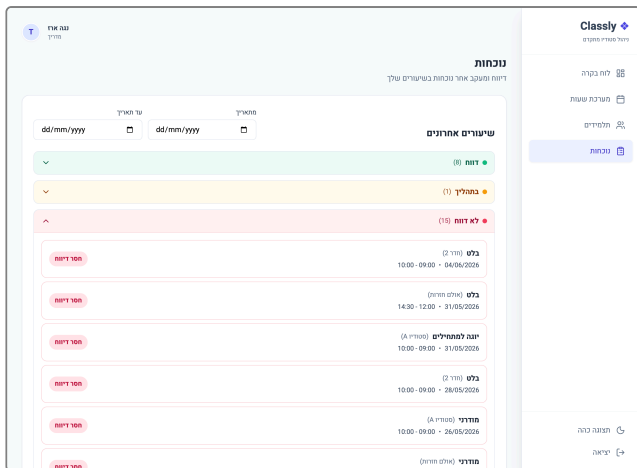
המדריך רואה ממשק ממוקד: לוח בקרה אישי, לוח השיעורים שלו, רשימת התלמידים בחוגיו, ומסך נוכחות. כל מסך מציג רק את הנתונים הרלוונטיים למדריך המחובר.



תמונה 8.8 — לוח השיעורים של המדריך



תמונה 8.7 — לוח הבקרה של המדריך



תמונה 8.9 — תלמידי המדריך

תמונה 8.10 — מסך הנוכחות של המדריך (שלושה מקטעים נפתחים)

ניהול ומעקב נוכחות (מדריך ומנהל)

מסך הנוכחות בנוי מרכיב תצוגה אחד משותף (components/common/AttendanceSections) המשרת הן את המדריך והן את המנהל, ללא כפילות קוד. השיעורים מחולקים לשלושה מקטעים נפתחים (אקורדיון שבו נפתח מקטע אחד בכל פעם) לפי סטטוס הדיווח: **דווח** / **בהחלף** / **לא דווח**. לחיצה על שיעור פותחת את חלון הדיווח (תמונה 8.13).

תצוגת המדריך (תמונה 8.10) מציגה רשימה שטוחה של שיעוריו בלבד. **תצוגת המנהל** (תמונה 8.11) מרכזת את נוכחות כל הסטודיו, עם סינון לפי סניף, מדריך, חדר, חוג, תלמיד וטווח תאריכים, והשיעורים בתוך כל מקטע מקובצים לפי סניף ← מדריך לסקירה נוחה.

אופיר גואטה

מנהל מערכת

ניהול סטודיו מתקדם

נוכחות

מעקב אחר דיווחי נוכחות בכל הסטודיו

סינון
⌵

סניף	מדריך	חדר	חוג	תלמיד
הכל	הכל	הכל	הכל	הכל

עד תאריך

מתאריך

- לוח בקרה
- מערכת שעות
- תלמידים
- נוכחות**
- תשלומים
- ניהול

סקירת נוכחות

השיעורים מקובצים לפי סניף ומדריך, ומסומנים לפי סטטוס הדיווח.

⌵
(8) דווח ●

⌵
(1) בתהליך ●

⬆️
(19) לא דווח ●

סינף פולג
📄

נגה ארז
👤

חסר דיווח

בלט (חדר 2)
 10:00 - 09:00 • 04/06/2026

חסר דיווח

בלט (חדר 2)
 10:00 - 09:00 • 28/05/2026

תצוגה כהה

יצאה →

A

אופיר גואטה

מנהל מערכת

Classly

ניהול סטודיו מתקדם

נכחות

מעקב אחר דיווחי נכחות בכל הסטודיו

סינון

סניף

הכל

מדריך

הכל

חדר

הכל

חוג

הכל

תלמיד

יעלה סיטרון

מתאריך

dd/mm/yyyy

עד תאריך

dd/mm/yyyy

נכחות התלמיד

השיעורים מסומנים לפי נכחות התלמיד שנבחר: נוכח / איחור / נעדר.

נוכח (11)

איחור / באישור (0)

נעדר (2)

בלט (אולם חזרות)

14:30 - 12:00 • 07/06/2026

נעדר

מודני (אולם חזרות)

16:00 - 14:30 • 07/06/2026

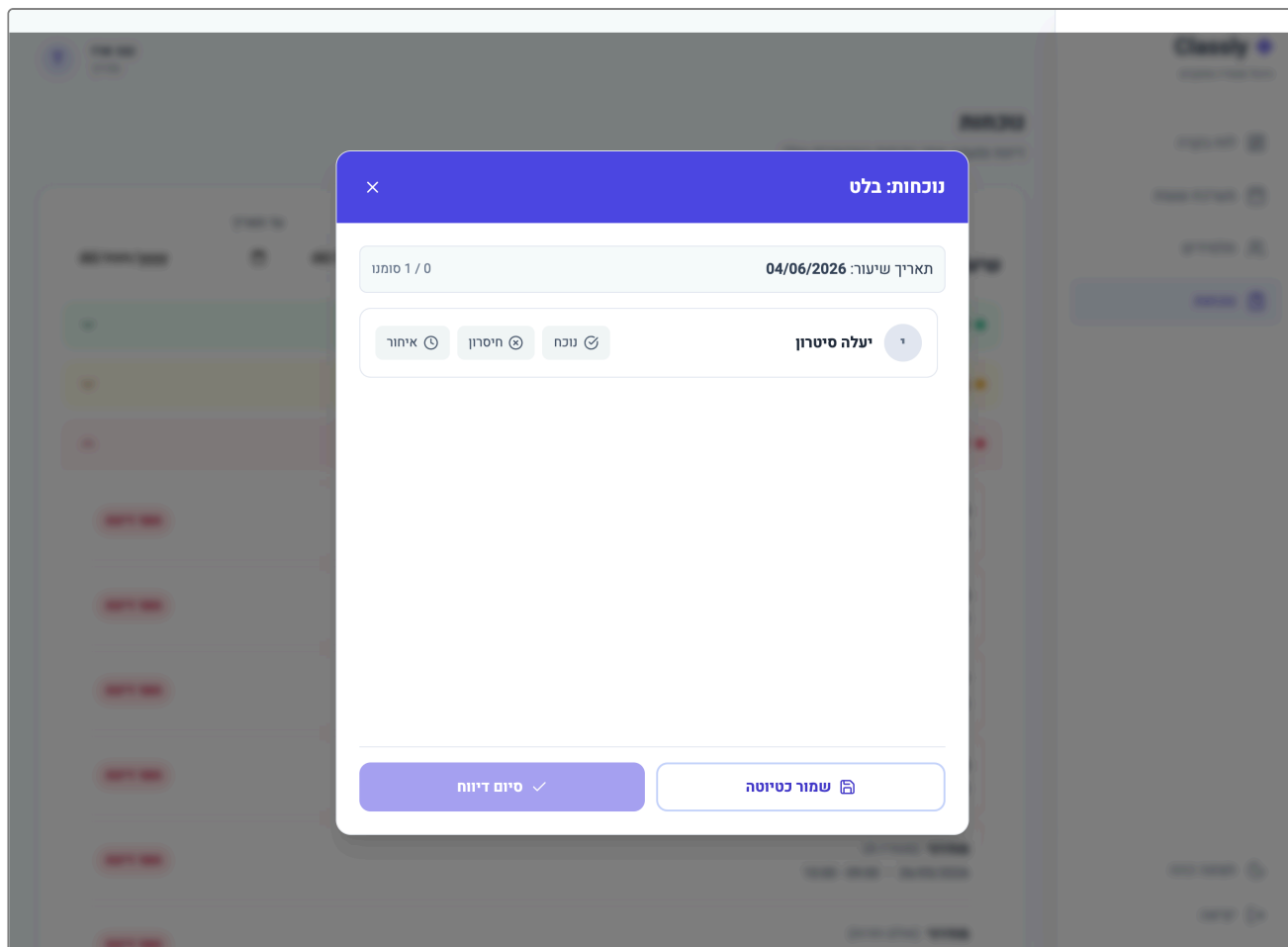
נעדר

תצוגה כהה

יציאה

תמונה 8.12 — סינון לפי תלמיד: רשימת השיעורים שבהם נכח/איחר/נעדר

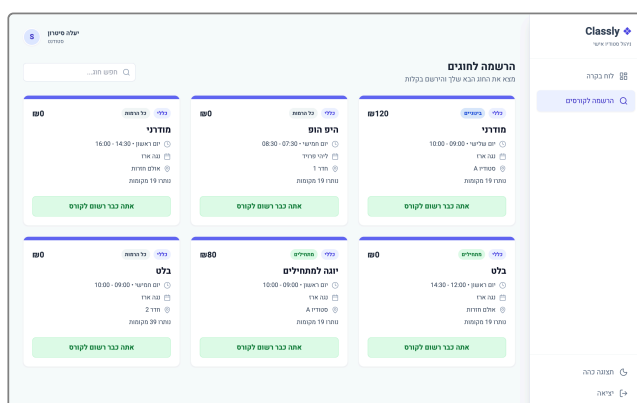
חלון הדיווח (משותף אף הוא) מציג את תלמידי החוג עם הסימונים נוכח / חיסרון / איחור. ניתן לשמור כטיוטה בכל עת, אך כפתור "סיום דיווח" מושבת עד שכל התלמידים סומנו. כמו כן, ניתן לפתוח דיווח לשיעור רק החל מ-15 דקות לפני תחילתו (שיעור עתידי מסומן כנעול), בעוד ששיעורי עבר נשארים פתוחים לדיווח.



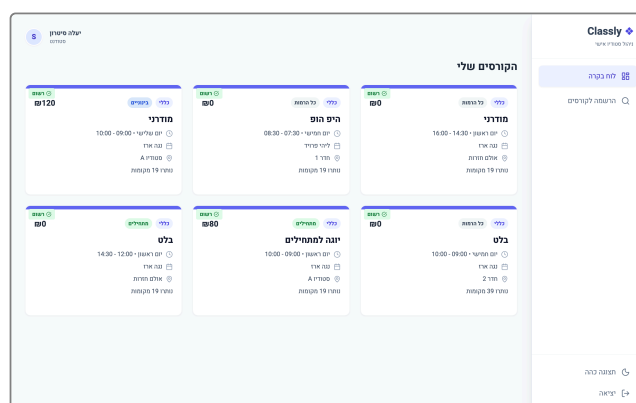
תמונה 8.13 — חלון דיווח הנוכחות ("סיום דיווח" מושבת עד שכל התלמידים סומנו)

מסכי התלמיד

התלמיד מקבל לוח אישי ומסך עיון בחוגים. בקטלוג מוצגים כל החוגים הזמינים — שם, מחיר, מועד, מדריך, חדר וקיבולת — והתלמיד יכול להירשם ולשלם. חוגים שהתלמיד כבר רשום אליהם מסומנים.



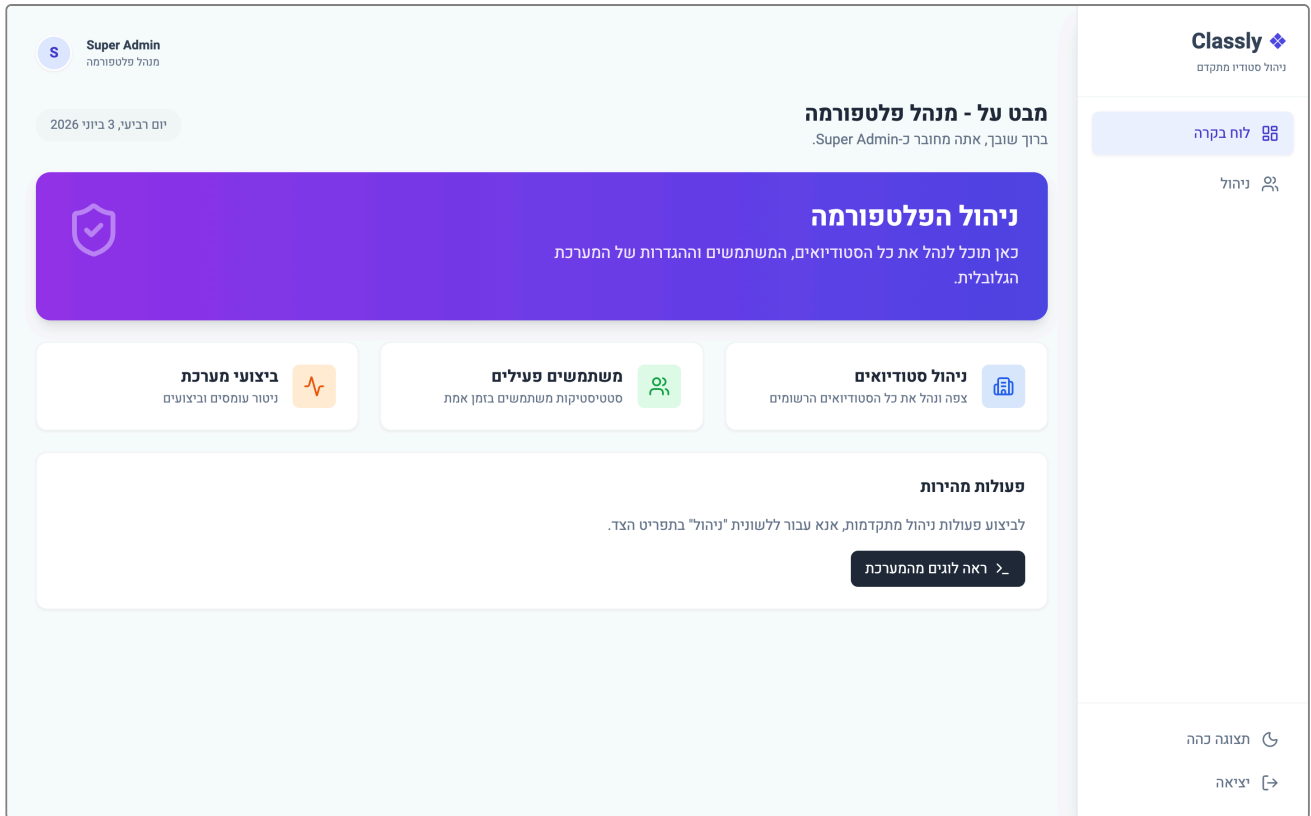
תמונה 8.15 — קטלוג החוגים (הרשמה)



תמונה 8.14 — לוח הבקרה של התלמיד

מנהל-על (פלטפורמה)

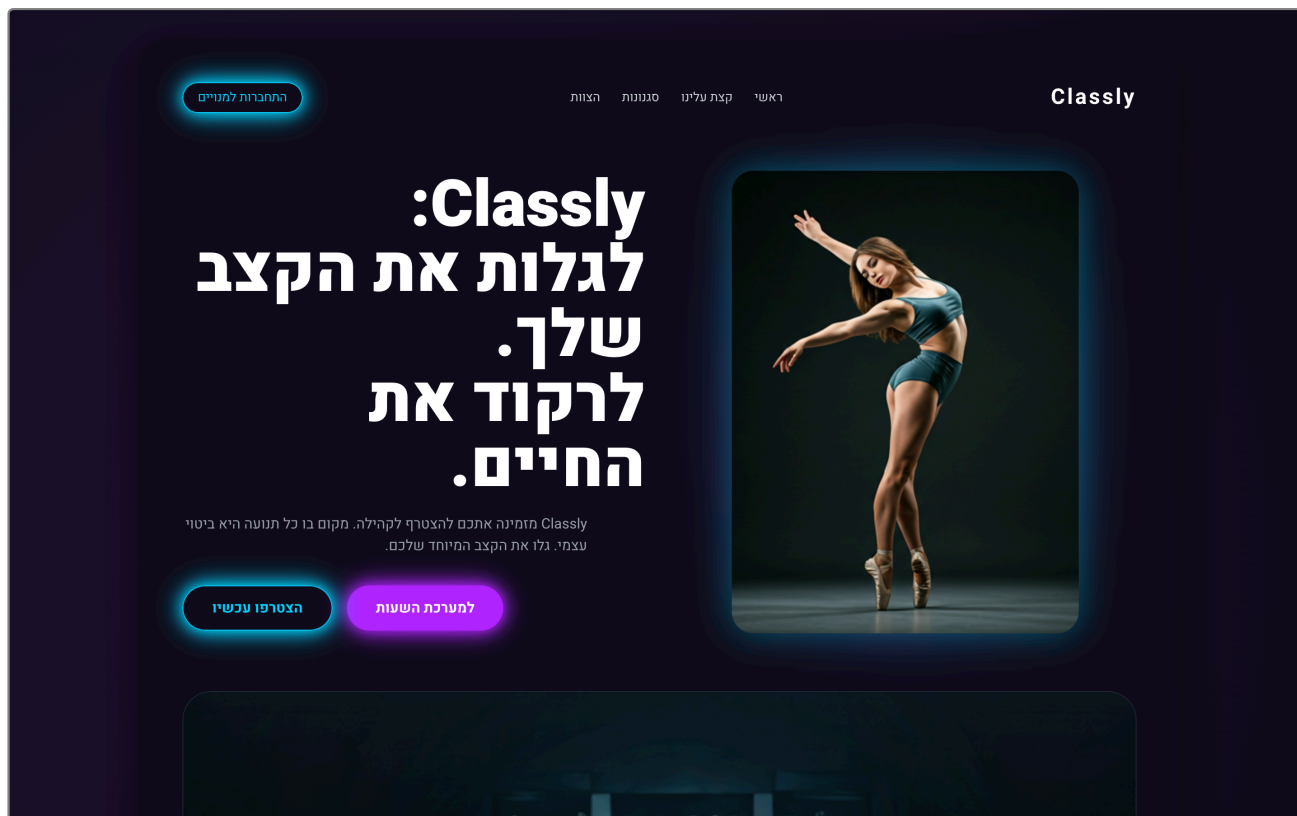
מנהל-העל מקבל תצוגת ניהול חוצה-סטודיואים: ניהול סטודיואים, משתמשים פעילים והגדרות מערכת.



תמונה 8.16 — לוח הבקרה של מנהל-העל

דף הנחיתה

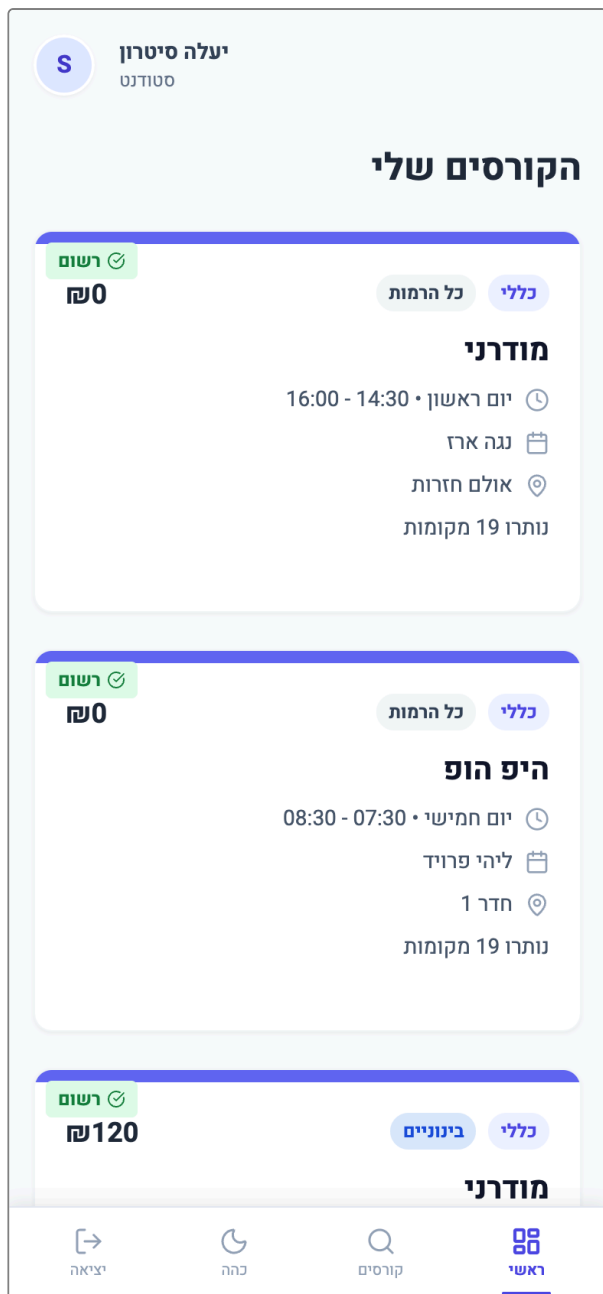
דף השיווק הציבורי של המוצר — Hero, מחירים, מדריכים, גלריה ושאלות נפוצות — עם קריאה להתחברות/הרשמה.



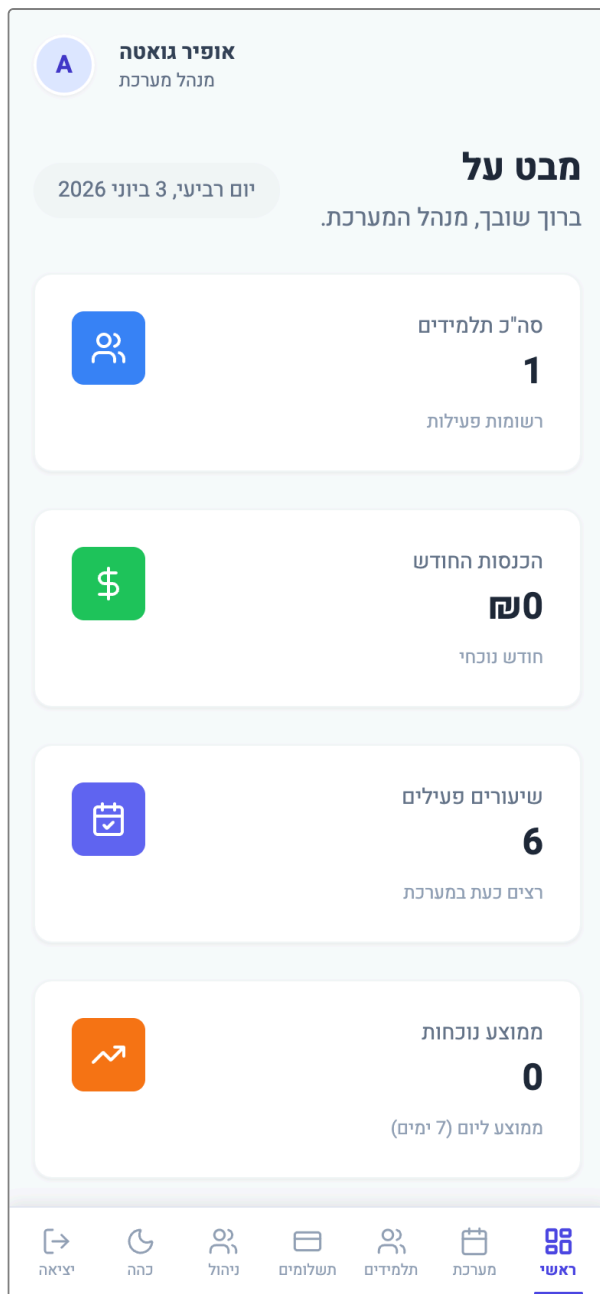
תמונה 8.17 — דף הנחיתה (Hero)

תצוגת מובייל (Mobile-first)

הממשק מותאם למסכים קטנים: במובייל סרגל הצד מתחלף בניווט תחתון (BottomNav), והתוכן מסתדר בעמודה אחת.



תמונה 8.19 — לוח תלמיד במובייל



תמונה 8.18 — לוח מנהל במובייל

8.2 ניהול מצב (State)

אין `store` גלובלי. מצב ההזדהות חי ב- `App.tsx` כ-`state` מקומי, ומועבר בהורשה (`props`) לקומפוננטות. עותק מסווג של המשתמש נשמר ב- `classly_user` (`localStorage`) לאתחול מהיר של ה-UI, אך מקור האמת הוא השרת — בעת טעינה נקראת `GET /api/auth/me` דרך העוגייה.

client/App.tsx

```
const [isAuthenticated, setIsAuthenticated] = useState(false);
const [currentUser, setCurrentUser] = useState<User | null>(null);
const [userRole, setUserRole] = useState<string>("STUDENT");
const [activeTab, setActiveTab] = useState("dashboard");

// /api/auth/me דרך HttpOnly-בדיקת הזדהות ראשונית - קוראת את עוגיית ה
useEffect(() => {
  const initialize = async () => {
    try {
      await AuthService.me(); // דורק 401 אם אין סשן תקף
      setIsAuthenticated(true);
      const cached = getStoredUser();
      if (cached) { setCurrentUser(cached); setUserRole(cached.role); }
      fetchUserRole(); // רענון פרופיל מהשרת ברקע
    } catch { /* 401 - נחיתה - מציגים דף נחיתה */ }
    finally { setLoading(false); }
  };
  initialize();
}, []);
```

תמונה 8.20 — אתחול מצב ההזדהות (App.tsx)

8.3 שכבת השירותים (api.ts)

מופע `axios` יחיד עם `withCredentials: true` (שולח את העוגייה בכל בקשה). `Interceptor` של בקשות רושם לוג, ו-`interceptor` של תשובות מטפל בשגיאת 401: אם פג תוקף הסשן, המשתמש מנותק והדף נטען מחדש. כל אובייקטי השירות (`AuthService`, `CourseService`, ...) הם עטיפות דקות מעליו.

client/services/api.ts

```
export const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  withCredentials: true, // שולח את עוגיות ה HttpOnly
});

// טיפול גלובלי ב-401 – טשן שפג תוקפו
apiClient.interceptors.response.use(
  (res) => res,
  (error) => {
    const isAuthRequest = AUTH_PATHS.some((p) => error.config?.url?.includes(p));
    if (error.response?.status === 401 && !isAuthRequest) {
      if (getStoredUser()) { removeStoredUser(); window.location.reload(); }
    }
    return Promise.reject(error);
  }
);

export const AuthService = {
  login: async (email, password) => {
    const res = await apiClient.post("/auth/login", { email, password });
    setStoredUser(res.data.user);
    return res.data;
  },
  me: () => apiClient.get("/auth/me").then((r) => r.data),
  logout: () => apiClient.post("/auth/logout").finally(removeStoredUser),
};
```

תמונה 8.21 — מופע axios יחיד עם interceptors ואובייקטי שירות (api.ts)

8.4 ניתוב והרשאות בצד הלקוח

במצב מחובר, התוכן נקבע לפי `activeTab`. הפונקציה `getComponentForTab` מחזירה קומפוננטה שונה לכל שילוב של טאב ותפקיד — וכך אותו "טאב" מציג מסך מותאם לכל סוג משתמש. **הסתרת טאבים היא קוסמטית בלבד**; האכיפה האמיתית היא בשרת.

client/App.tsx

```
const getComponentForTab = (tab: string) => {
  switch (tab) {
    case "dashboard":
      if (userRole === "SUPER_ADMIN") return <SuperAdminDashboard />;
      if (userRole === "ADMIN") return <Dashboard onTabChange={setActiveTab} />;
      if (userRole === "INSTRUCTOR") return <InstructorDashboard />;
      return <StudentDashboard />;
    case "schedule":
      return userRole === "ADMIN" ? <ClassSchedule /> : <InstructorSchedule />;
    case "payments":
      if (userRole === "ADMIN") return <Payments />;
      return <div>הרשאה איננה הרשאה</div>;
    case "browse": return <BrowseCourses />; // STUDENT
    // ... students, attendance, administration
  }
};
```

תמונה 8.22 — בחירת קומפוננטה לפי טאב ותפקיד (App.tsx)

פריטי הסרגל עצמם מסוגנים לפי תפקיד — כל פריט מגדיר אילו תפקידים רשאים לראותו:

client/components/Sidebar.tsx

```
const allMenuItems = [
  { id: "dashboard", label: "לוח בקרה", roles: ["ADMIN", "INSTRUCTOR", "STUDENT", "SUPER_ADMIN"] },
  { id: "browse", label: "הרשמה לקורסים", roles: ["STUDENT"] },
  { id: "schedule", label: "מערכת שעות", roles: ["ADMIN", "INSTRUCTOR"] },
  { id: "students", label: "תלמידים", roles: ["ADMIN", "INSTRUCTOR"] },
  { id: "attendance", label: "נוכחות", roles: ["INSTRUCTOR"] },
  { id: "payments", label: "תשלומים", roles: ["ADMIN"] },
  { id: "administration", label: "ניהול", roles: ["ADMIN", "SUPER_ADMIN"] },
];
const menuItems = allMenuItems.filter((item) => item.roles.includes(userRole));
```

תמונה 8.23 — סינון פריטי הניווט לפי תפקיד (Sidebar.tsx)

8.5 ארגון קומפוננטות — "חכמה" מול "טיפשה"

הקומפוננטות מאורגנות לפי תפקיד (admin/, instructor/, student/, super-admin/), ומשתמשות בהפרדה בין קומפוננטה "חכמה" (מנהלת מצב ושולפת נתונים — למשל StudentManagement) לבין קומפוננטה "טיפשה"/מוצגת (מקבלת נתונים ב-props ומציגה בלבד — למשל StatCard, ClassCard). כך נמנעת חזרה על קוד ונשמרת גמישות.

client/components/common/StatCard.tsx

```
// ומציגה בלבד props-קומפוננטה מוצגת ("טיפשה") - מקבלת נתונים ב
export const StatCard: React.FC<StatCardProps> = ({ title, value, subtext, icon: Icon,
color }) => (
  <div className="bg-white p-6 rounded-xl shadow-sm border dark:bg-slate-900">
    <div>
      <p className="text-sm text-slate-500">{title}</p>
      <h3 className="text-2xl font-bold">{value}</h3>
      <p className="text-xs mt-2">{subtext}</p>
    </div>
    <div className={`p-3 rounded-lg ${color}`}><Icon className="text-white" size={24}</div>
  </div>
</div>
);
```

תמונה 8.24 — קומפוננטה מוצגת גנרית (StatCard.tsx)

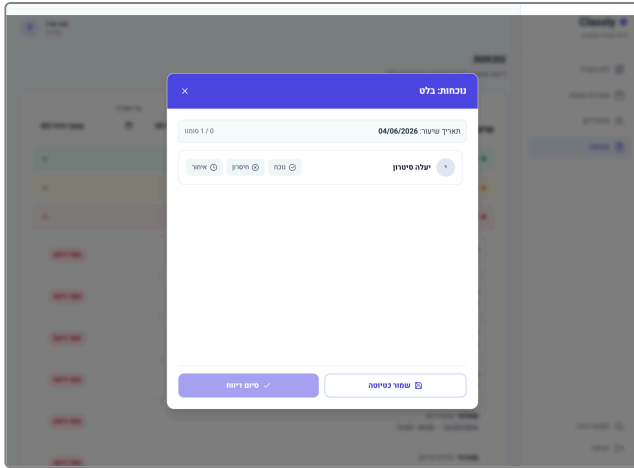
8.6 קומפוננטות גנריות משותפות ומודלים

בתיקיית components/common/ מרכזות קומפוננטות לשימוש חוזר. BaseModal היא תשתית אחידה לכל החלונות הקופצים במערכת — היא נועלת גלילה, מציגה כותרת אחידה, ומקבלת תוכן וכפתורי תחתית.

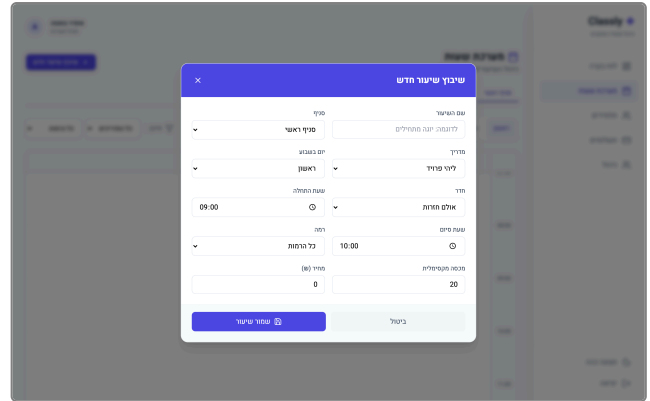
client/components/common/BaseModal.tsx

```
export const BaseModal: React.FC<BaseModalProps> = ({ isOpen, onClose, title,
children, footer }) => {
  useEffect(() => { // נעילת גלילת הרקע כשהמודל פתוח
    document.body.style.overflow = isOpen ? "hidden" : "auto";
  }, [isOpen]);
  if (!isOpen) return null;
  return (
    <div className="fixed inset-0 z-50 flex items-center justify-center bg-black/50">
      <div className="bg-white rounded-2xl shadow-xl max-w-2xl dark:bg-slate-900">
        <div className="bg-indigo-600 p-6 text-white">{title}</div>
        <div className="p-6 overflow-y-auto">{children}</div>
        {footer && <div className="p-6 border-t">{footer}</div>}
      </div>
    </div>
  );
};
```

תמונה 8.25 — תשתית מודל אחידה (BaseModal.tsx)



תמונה 8.27 — מודל סימון נוכחות



תמונה 8.26 — מודל "שיבוץ שיעור חדש"

8.7 אינטגרציית Stripe Elements (צד לקוח)

בצד הלקוח נטענת ספריית Stripe, וטופס התשלום מוצג באמצעות `<PaymentElement />` בתוך `<Elements>`. פרטי הכרטיס נאספים ומאומתים מול Stripe ישירות — הם אינם נוגעים בשרת שלנו. אישור התשלום נעשה עם `clientSecret` שהתקבל מהשרת.

client/components/admin/Payments.tsx

```
const stripePromise = loadStripe(import.meta.env.VITE_STRIPE_PUBLISHABLE_KEY || "");

const handleSubmit = async (e) => {
  e.preventDefault();
  const { error, paymentIntent } = await stripe.confirmPayment({
    elements,
    confirmParams: { return_url: window.location.origin + "/payment/success" },
    redirect: "if_required",
  });
  if (paymentIntent?.status === "succeeded") onSuccess();
};

// JSX: טופס התשלום
<Elements stripe={stripePromise} options={{ clientSecret }}>
  <PaymentElement />
  <button>בצע תשלום מאובטח</button>
</Elements>
```

תמונה 8.28 — איסוף תשלום מאובטח עם Stripe Elements (Payments.tsx)

8.8 עיצוב RTL, Tailwind ומצב כהה

כל הממשק מוגדר `dir="rtl"` בשורש האפליקציה, והעיצוב נעשה עם **TailwindCSS** תוך העדפת `utilities` לוגיים (`me-*`, `ms-*`) על פני כיווניים. המערכת תומכת במצב כהה (Dark Mode) הניתן להחלפה מהסרגל, באמצעות מחלקות `dark:` של Tailwind (כפי שניתן לראות בקטעי הקוד שלעיל).

אבטחה בצד לקוח

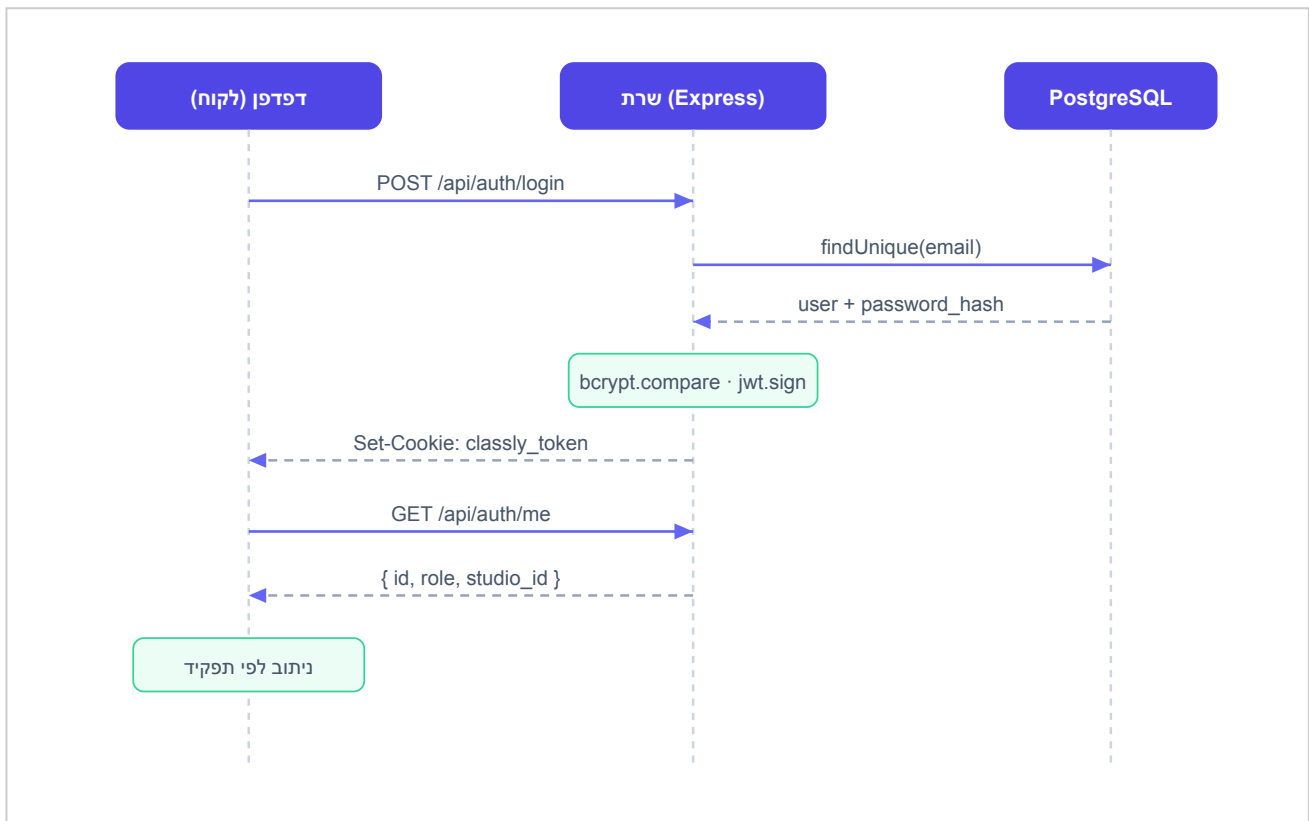
הסתרת טאבים וכפתורים לפי תפקיד היא לנוחות התצוגה בלבד. כל פעולה רגישה נאכפת מחדש בשרת — לעולם אין להסתמך על מצב הלקוח לצורכי אבטחה.

9. תהליכים מרכזיים במערכת

תרשימי רצף (Sequence) של התהליכים העיקריים

9.1 תהליך התחברות

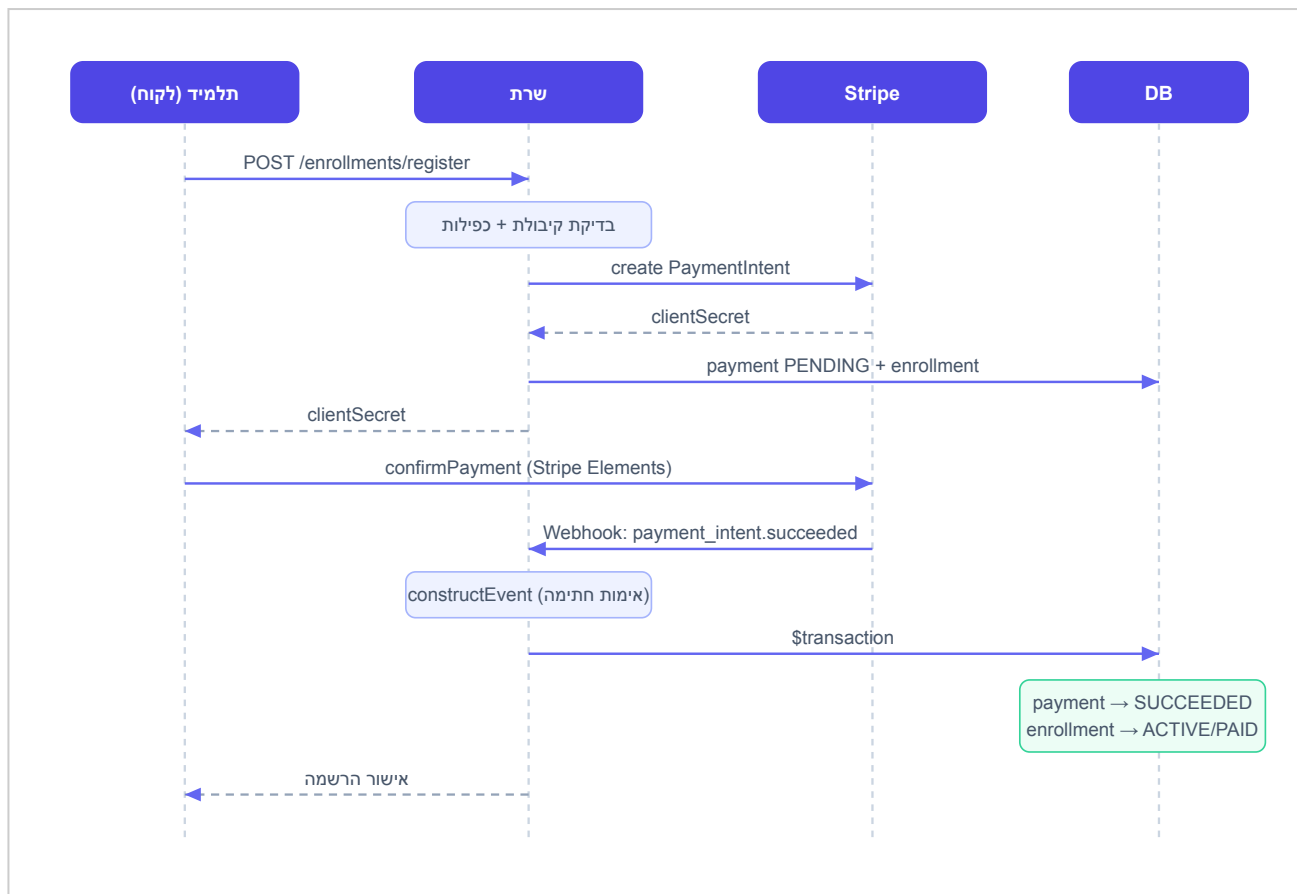
המשתמש מזין פרטים; השרת מאמת מול ה-DB, חותם JWT ושומר אותו בעוגיית HttpOnly; לאחר מכן הלקוח קורא ל- `/api/auth/me` ומנתב לפי התפקיד.



תרשים 9.1 — תהליך התחברות וניתוב לפי תפקיד

9.2 תהליך הרשמה לחוג ותשלום

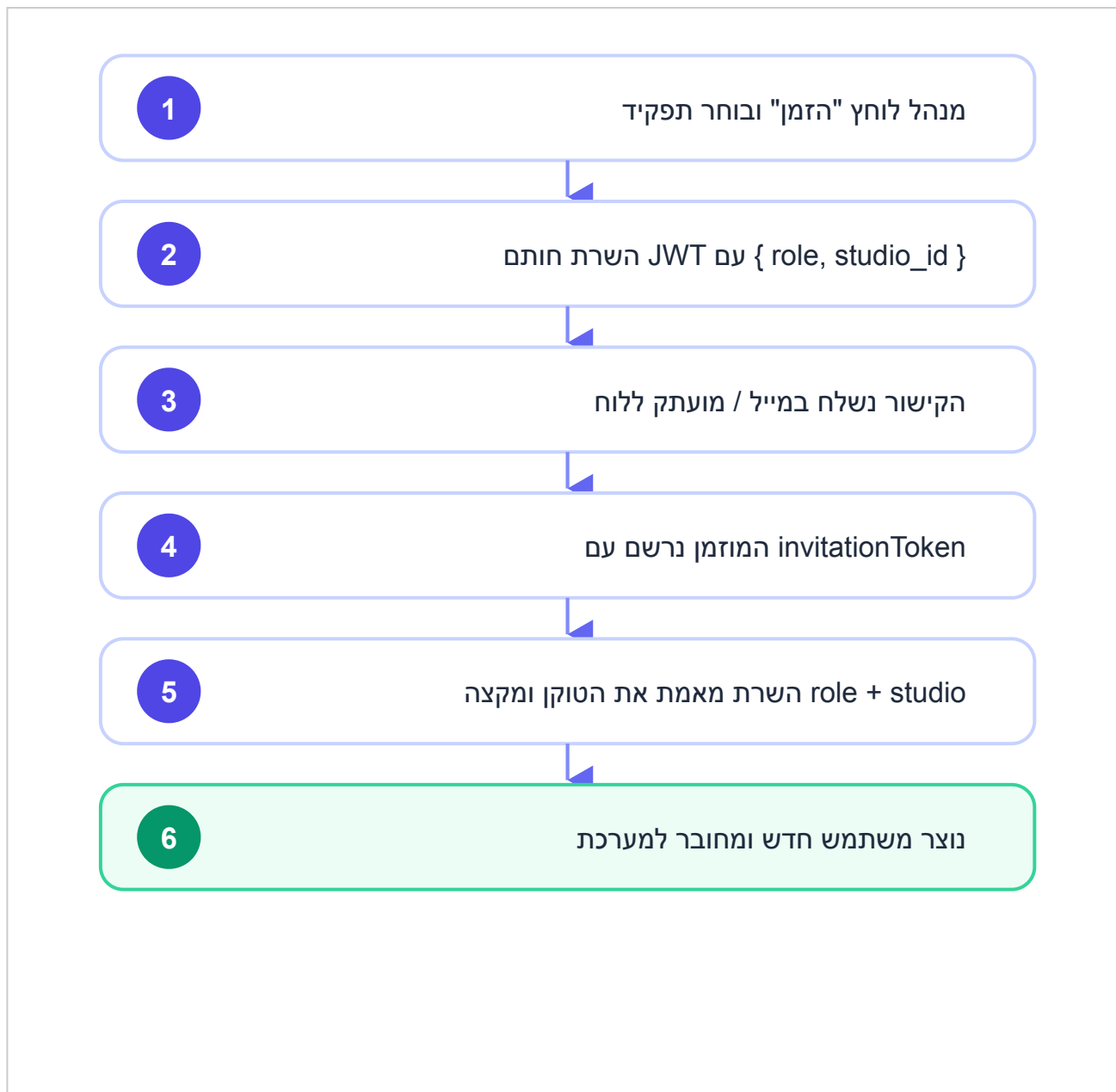
התהליך המרכזי של המערכת: התלמיד נרשם, נוצרת כוונת תשלום, הוא משלם דרך Stripe, ולבסוף Webhook מאמת-חתימה מפעיל את ההרשמה בתוך טרנזקציה — כך שגם אם הדפדפן נסגר, ההרשמה תאושר.



תרשים 9.2 — תהליך הרשמה לחוג ותשלום (כולל Webhook אידימפוטנטי)

9.3 תהליך הזמנת משתמש

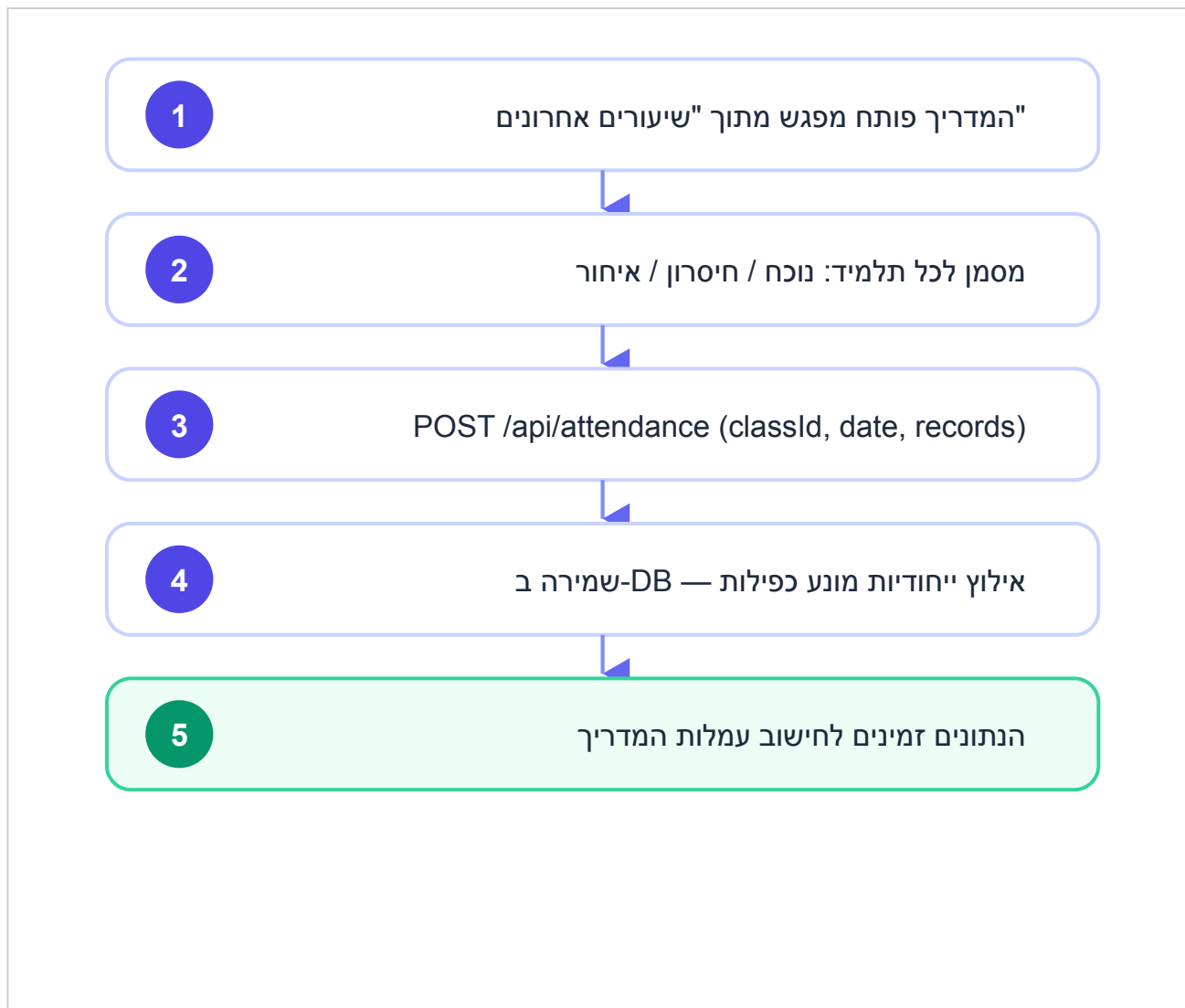
מנהל מזמין מדריך/תלמיד; ההזמנה היא JWT הנושא `role` ו-`studio_id`. בעת הרשמה עם הטוקן, התפקיד והסטודיו מוקצים אוטומטית.



תרשים 9.3 — תהליך הזמנת משתמש

9.4 תהליך סימון נוכחות ומעקב

המדריך מסמן נוכחות למפגש; אילוף ייחודיות על (enrollment_id, session_date) מונע סימון כפול לאותו מפגש, והנתונים משמשים בהמשך לחישוב עמלות. דיווח נפתח רק החל מ-15 דקות לפני תחילת השיעור; ניתן לשמור כטייטה (מצב IN_PROGRESS) בכל שלב, ו"סיום דיווח" (COMPLETED) מתאפשר רק לאחר שכל התלמידים סומנו.

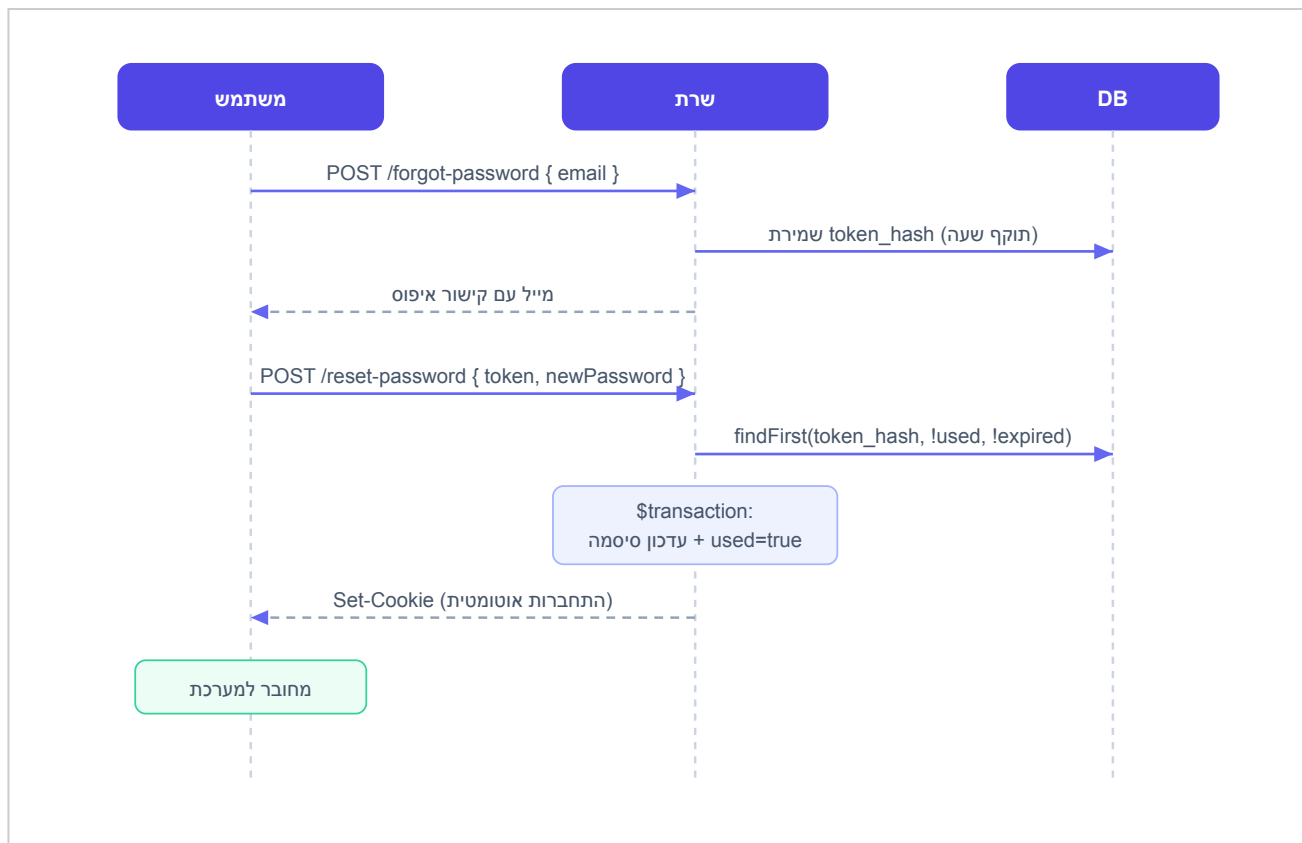


תרשים 9.4 — תהליך סימון נוכחות

סקירת המנהל. בנוסף לדיווח המדריך, המנהל צופה בנוכחות כלל הסטודיו דרך נקודת קצה אחת (`GET /api/attendance/overview`) שמחזירה את אותם נתונים בשני מצבים: ברירת מחדל — תצוגת מפגשים מקובצת לפי סניף ומדריך; ובהעברת `studentId` — תצוגה ממוקדת-תלמיד. שני המסכים (מדריך ומנהל) צורכים את אותו רכיב תצוגה משותף, כמתואר בפרק "צד לקוח" (תמונות 8.10–8.13).

9.5 תהליך איפוס סיסמה

נשמר רק `hash` של הטוקן (לעולם לא הטוקן הגולמי), עם תוקף שעה. האיפוס עצמו והסימון "נוצל" מתבצעים בתוך טרנזקציה, ולאחריו המשתמש מחובר אוטומטית.



תרשים 9.5 — תהליך איפוס סיסמה

10. סיכום ומסקנות

פרויקט **Classly** פיתח את יכולתנו לתכנן ולממש מערכת ווב מלאה (full-stack) מהאפס — מהקמת בסיס נתונים ותכנון ברמת ארכיטקטורה, דרך פיתוח צד שרת וצד לקוח והחיבור ביניהם, ועד הרצת המערכת כולה בסביבת קונטיינרים אחת.

במהלך הפרויקט התמודדנו עם מספר אתגרים מרכזיים:

- **בידוד רב-דיירי** — הבטחה שכל סטודיו רואה אך ורק את נתוניו, באמצעות סינון עקבי לפי `studio_id` בכל שאילתה.
- **תשלומים מקוונים** — אינטגרציה עם Stripe, כולל אימות חתימת Webhook על גוף גולמי ושמירה על אידמפוטנטיות מול שליחות חוזרות.
- **עקביות נתונים** — שימוש בטרנזקציות כדי שפעולות מרובות-טבלאות (הרשמה + תשלום) יתבצעו כיחידה אטומית.
- **ממשק עברי RTL** ומותאם מובייל, עם תמיכה במצב כהה.
- **תזמור Docker** — הרצת כל השירותים (DB, שרת, לקוח, לוגים) בפקודה אחת, בסביבה זהה לפיתוח ולייצור.

למדנו לעומק טכנולוגיות מודרניות — TypeScript, Prisma, React, Stripe, Docker ו-Loki/Grafana — וכן עקרונות הנדסת תוכנה כמו ארכיטקטורת שכבות, הפרדת אחריות, קומפוננטות גנריות ואבטחה כברירת מחדל.

שיפורים עתידיים

כחלק מהסתכלות קדימה, זיהינו מספר כיווני שיפור שיחזקו את המערכת:

שיפור	תועלת
שער בדיקות ב-CI	הרצת typecheck ובדיקות אוטומטית בכל PR לפני מיזוג ל-main.
מודול <i>TenantScope</i>	ריכוז סינון ה- <code>studio_id</code> במקום אחד — כך אי-אפשר "לשכוח" אותו בשאילתה.
טבלת <code>webhook_events</code>	מניעת עיבוד כפול של אירועי Stripe ברמת מזהה האירוע (dedup מלא).
Refresh Tokens	חידוש ששן שקוף ללא טעינה מחדש של הדף לאחר פקיעת הטוקן.
אינדקסים ב-DB	אינדקסים על <code>studio_id</code> , <code>status</code> ועמודות זמן — לשיפור ביצועים בנפח גדול.
i18n	חילוץ מחרוזות עברית והוספת אנגלית להרחבת קהל היעד.

לסיכום, Classly מדגימה כיצד ניתן לבנות פלטפורמת SaaS רב-דיירית, מאובטחת ונוחה לשימוש, תוך שימוש בכלים מודרניים ובעקרונות הנדסיים מבוססים — תוצר שאנו גאים בו ושממנו למדנו רבות.

11. ביבליוגרפיה

תיעוד רשמי ומקורות מידע

- /React — <https://react.dev>
- /Vite — <https://vitejs.dev>
- /TypeScript — <https://www.typescriptlang.org/docs>
- /Express — <https://expressjs.com>
- Prisma — <https://www.prisma.io/docs>
- /PostgreSQL — <https://www.postgresql.org/docs>
- Stripe — <https://stripe.com/docs>
- Stripe Elements (React) — <https://stripe.com/docs/stripe-js/react>
- /Docker — <https://docs.docker.com>
- TailwindCSS — <https://tailwindcss.com/docs>
- /JSON Web Tokens — <https://jwt.io>
- /Zod — <https://zod.dev>
- /Pino (Logging) — <https://getpino.io>
- /Grafana Loki — <https://grafana.com/oss/loki>
- /Nodemailer — <https://nodemailer.com>
- bcrypt — <https://www.npmjs.com/package/bcryptjs>
- /Helmet — <https://helmetjs.github.io>
- /Axios — <https://axios-http.com>
- /Recharts — <https://recharts.org>